

Namo Roadmap

Date: 20260608

Design philosophy

Analytical freedom

Namo's foundational design principle is that every key in a row/record/hash is a dimension. There is no distinction between 'axes' and 'values'. This means coordinates are values and values are coordinates.

Namo infers everything from the hash keys. There is no configuration step, no reshaping, no schema declaration. An array of hashes from a database, a CSV, a JSON API, or a YAML file goes straight into Namu and every key is immediately selectable, projectable, and usable in formulae.

This simplification is powerful because it eliminates the distinction that every other tool requires to be made at ingestion time. Analytical decisions belong to the analyst, to be made later during analysis. Every other library in this space requires the user to commit to analytical decisions at ingestion time — which columns are the index, which are data variables, what types they have, when computation runs, where derived values come from. Each of those commitments narrows what the user can later do with the loaded data.

The structure of an analysis — which dimensions are grouped by, which are aggregated over, which serve as identifiers and which as measurements — are determined by the questions asked of the data, not the data as structured. The same dataset answers different questions, and Namu doesn't presage any at ingestion.

A hash {symbol: 'BHP', date: '2025-01-01', close: 42.5} has three dimensions, all equally first-class. You can select on close the same way you select on symbol — `namu[close: 42.5]` is just as valid as `namu[symbol: 'BHP']`. You can project to symbol and date and drop close, or project to close alone. No dimension is privileged. No field is "just data."

`coordinates.close` and `values.close` are the same data viewed two ways — `coordinates` is `values` deduplicated. The shallow API distinction reflects a deep equivalence: every value is a coordinate of its dimension, every coordinate is a value present in the data. Whether you treat `close` as an axis to group by or a quantity to compute over is decided at query time.

Other libraries force the decision earlier. Pandas requires one to choose which columns are the index. xarray requires one to declare dimensions, coordinates, and data variables separately. In each case, the user must understand their data's analytical structure before they can load it — and once loaded, restructuring is awkward.

Type agnosticism

The kinds of computation you'll do are also deferred. Namu doesn't privilege numbers. A formula can concatenate strings, produce status labels, or generate alert messages as naturally as it can compute ratios:

```
e.label = proc{ "#{symbol} ({exchange})" }
e.status = proc{ volume > 0 ? 'active' : 'suspended' }
e.alert = proc{ "#{symbol}: #{action} at #{close}" }
```

These are just as valid as `proc{ close / book_value }`. The formula mechanism resolves names and calls procs. What the proc does with the values — arithmetic, string manipulation, date logic, pattern matching — is Ruby's business, not Namu's.

This means Namu can handle datasets that aren't numerical at all: customer records, event logs, text corpora, legal documents with categorised metadata, survey responses. Anything expressible as an array of hashes. The selection, projection, contraction, composition, and set operators all work on text, dates, booleans, and arbitrary objects the same way they work on numbers. `namu[status: 'active']` works because `status` is a dimension like any other, and strings are values like any other.

No other tool in this space offers this. Pandas comes closest — DataFrames can hold mixed types — but its computation model (`df['col'].rolling().mean()`) assumes numeric columns. `xarray` is explicitly numerical. `Quantix` and `Improv` are spreadsheets. `Polars` is a columnar computation engine optimised for numeric and string operations but not arbitrary Ruby objects. `Namo`'s formula mechanism works on whatever Ruby can work on, which is everything.

`Namo` is not a computation engine or a multidimensional spreadsheet. It is a multidimensional database. The distinction matters: spreadsheets and numerical tools like `Quantix`, `Improv`, `xarray`, `Pandas`, and `Polars` are fundamentally organised around numbers. Their dimensions exist to label numerical data. Text in a dimension is a category label, not something you compute on.

Live computation objects

Similarly to how `Namo` treats all dimensions as both data and coordinates, formulae are treated as derived dimensions. `row.close` and `row.earnings_yield` resolve through the same mechanism. The consumer doesn't need to know or care whether a dimension is data or a derived formula. They're all just names with values.

A `Namo` holds data and computation together as a single object whose computed values are always current with respect to its data. Stored values and formulae are accessed through one interface; both reflect current state on every access.

`row.close` returns the stored value. `row.return` invokes the proc and returns the computed value. Same syntax, same semantics from the consumer's perspective. The fact that one is stored and one is computed is an implementation detail of the dimension, not a fact about how it's accessed.

When the underlying data changes — new rows append, existing rows are updated, the backing store delivers different content — every computed value reflects the change without reconciliation. Pull-based reactivity through laziness: each access recomputes from current state, so there's no stale-snapshot problem to solve.

This holds across the formula trajectory: single-row formulae shipped in 0.1.0, cross-row formulae planned in 0.15.0, parameterised formulae in 0.16.0. Same mechanism throughout, same currentness property regardless of formula complexity.

Other tabular libraries produce snapshots. `Pandas`, `Polars`, `dplyr`, and `DataFrames.jl` all materialise computed values into data columns, severing them from the computation that produced them. Spreadsheets like `Excel` and `Quantix` attach formulae to cells but operate on them through a different interface than data. `Namo` treats them identically across the entire algebra of operations it provides.

Unified treatment of data and derived dimensions

Following from live computation: every operation `Namo` provides treats data dimensions and derived dimensions equivalently. Selection, projection, contraction, composition, set operators, comparison operators, pattern-matching against schemas — none of them distinguish how a dimension's values are produced.

```
namo[:revenue]                # projection – works whether a dimension is data or derived
namo[revenue: 1000..2000]     # selection – works whether a dimension is data or derived
namo - returned_items         # set operation – formulae carry through
namo * fundamentals           # composition – formulae merge by rule
case other; when namo; ...    # pattern-match (Namo#===) – checks data and derived dimensions together
```

Analytical structure can be added or removed without re-ingesting. A user can attach `:return`, `:sma_20`, and `:signal` formulae to an existing `Namo` at any time; subsequent queries treat them as first-class dimensions. They can be removed equally freely. The data layer is untouched; the analytical layer grows and shrinks at the analyst's discretion.

This is what “manipulating derived dimensions as data” means: not a metaphor, but a description of the architecture. Liveness is the mechanism; unified interface is the consequence.

Self-contained formulae

For the unified interface to work, formulae need to be self-contained — pure functions of the row (or (row, namo) for cross-row formulae). They depend on their inputs, not on the environment they were defined in.

```
# Good – pure function of row
e[:return] = proc{|row| row[:close] / row[:previous_close] - 1}

# Bad – depends on external state
@discount_rate = 0.05
e[:discounted] = proc{|row| row[:value] / (1 + @discount_rate)}
```

The self-contained pattern is what makes formulae portable. A Namo can be serialised, sent to another process or another machine, and its formulae will produce the same results because they don't depend on the environment they were defined in. The pattern is also what makes formulae safe to compose — two Namos can be merged by `*` and their formulae coexist because neither references anything outside its inputs.

This isn't enforced by the language (Ruby procs can capture whatever they like), but it's the convention Namo's design depends on. Future versions may move toward stricter enforcement or toward formula representations that don't allow external capture at all.

Algebraic operations on whole Namos

A Namo is a value in an algebra of operations, not just a collection to iterate. Set operators (`+`, `-`, `&`, `|`, `^`) combine Namos as sets of rows. Composition operators (`*`, `**`, `/`) combine Namos with different dimensions. Comparison operators (`==`, `<`, `<=`, `>`, `>=`) ask structural questions about subset relations. `===` asks pattern-match questions about analytical shape.

```
combined = ohlcv * fundamentals    # join on shared dimensions
held      = portfolio & universe    # rows present in both
losers    = today - yesterday       # rows new today
```

These operators take Namos as inputs and produce Namos as outputs. The result of every operator is itself queryable, composable, and operable. The algebra closes.

The shape of an analytical pipeline is not pre-committed to. Operators compose in whatever order makes sense for the question being asked. No other DataFrame library exposes its operations as algebraic operators. Pandas has methods (`merge`, `concat`, `drop_duplicates`); R has function syntax (`bind_rows()`, `inner_join()`); Polars has fluent method chains. Namo treats the operations as first-class operators on the language level, which makes pipelines compact, readable, and uniform.

Composition with surrounding systems

A Namo is passive. It holds state and answers queries. It does not provide subscription mechanisms, event propagation, dependency tracking, or change-detection infrastructure. The reactivity model is the analyst's decision, not Namo's.

Real-time systems built on Namo wire push-based change-detection to the Namo's mutation side and pull-based consumers to the Namo's query side. The Namo doesn't need to know about either; the surrounding system composes them.

```
# Push side: external mechanism updates the Namo
websocket.on_tick{|tick| analysis.append(tick); notifier.broadcast(:updated)}

# Pull side: consumer queries when notified
notifier.subscribe(:updated){|_| update_dashboard(analysis[criteria])}
```

Three components, three responsibilities, clean interfaces. The same passive Namo works in batch reports (pull only, no notifier), interactive analysis (manual queries), polling systems (pull on a timer), and real-time dashboards (push triggers, pull queries) without modification.

Reactive frameworks like RxJS push values through subscription graphs. Namo provides the live computation property that makes such frameworks valuable, but leaves the reactivity infrastructure to the user — to be chosen and composed with whatever the rest of the system already uses. The Namo's job is to be a current-valued data container; everything else is composable around it.

Portability of analytical artefacts

Following from self-contained formulae and the unified interface: a Namo is a portable analytical artefact. Data plus computation plus structure travel together. Serialise a Namo, send it to a colleague, and they have everything needed to query it, extend it, and recompute its derived values.

This is the property notebooks try to provide and fail at. Jupyter notebooks intermix code cells, output cells, and environment state — the artefact you receive is not deterministically re-runnable. A Namo is different: the data is structurally present, the formulae are self-contained procs, and the receiver can call any query operation on it without depending on a particular Ruby environment, package set, or execution order.

This isn't yet a shipped feature — serialisation lands later in 1.x — but the design enables it. Every other principle in this section contributes: self-contained formulae are portable; unified interface means the receiver doesn't need to know what's data vs derived; type agnosticism means the format doesn't have to encode complex type systems; analytical structure at query time means the structure is in the data itself, not in a separate schema.

A Namo is a small, complete, self-describing analytical object. Pandas DataFrame plus the script that produced its computed columns. Excel workbook plus the ability to be queried programmatically. Jupyter notebook minus the bullshit.

Current state: 0.14.0

0.0.0 (2026-03-15): Initial release

Instantiation from an array of hashes. Namo infers dimension names from hash keys and extracts unique coordinate values per dimension automatically. Selection via keyword arguments in [], supporting single values, arrays, and ranges.

```
sales = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40}
])

sales.dimensions    # => [:product, :quarter, :price, :quantity]
sales.coordinates   # => {product: ['Widget', 'Gadget'], quarter: ['Q1', 'Q2'], ...}
sales[product: 'Widget']      # single value
sales[quarter: ['Q1', 'Q2']]  # array
sales[price: 10.0..20.0]      # range
```

This release established the foundational insight: every key is a dimension, every value is a coordinate. No configuration, no schema, no reshaping step.

0.1.0 (2026-03-28): Formulae and projection

Formulae via []=. A formula is a proc assigned to a name. It receives a Row object and resolves named references against row data or other formulae. Formulae compose — a formula can reference another formula, and the dependency chain resolves lazily through Row.

Projection via positional symbol arguments in []. Selection and projection can be combined in a single call or chained.

```
sales[:revenue] = proc{|r| r[:price] * r[:quantity]}
sales[:profit] = proc{|r| r[:revenue] - r[:cost]}
```

```
sales[:product, :quarter, :revenue]      # projection
sales[:product, :revenue, product: 'Widget'] # projection + selection
sales[product: 'Widget'][:revenue]       # chained
```

Also introduced `to_a` for extracting data as an array of hashes, the `Row` class for formula resolution, and `attr_accessor :data, :formulae`.

0.2.0 (2026-04-14): Enumerable

Namo includes Enumerable. The `each` method yields `Row` objects (not raw hashes), so Enumerable methods like `map`, `select`, `reduce`, `min_by`, and `flat_map` all see formulae as though they were data.

```
sales[:revenue] = proc{|r| r[:price] * r[:quantity]}
total = sales.reduce(0){|sum, row| sum + row[:revenue]}
cheapest = sales.min_by{|row| row[:price]}
```

Selection logic moved from `Namo` to `Row#match?`. Added `Row#to_h`. `each` returns an Enumerator when no block is given.

0.3.0 (2026-04-15): Contraction

Contraction via negated symbols in []. The unary minus on a symbol (`-:price`) produces a `NegatedDimension`, which [] recognises as “remove this dimension from the result.” The complement of projection — projection says what to keep, contraction says what to remove.

```
sales[-:price, -:quantity]      # remove dimensions
sales[-:price, -:quantity, product: 'Widget'] # contraction + selection
```

Raises `ArgumentError` when mixing projection and contraction in the same call. Formulae carry through contraction. `NegatedDimension` and `Symbol#-@` introduced as supporting classes.

`Row` extracted to its own file (`Namo/Row.rb`). Tests split into per-class files.

0.4.0 (2026-04-15): Concatenation and row removal

The first two row-axis set operators.

`+` concatenates two `Namos` with matching dimensions. Appends rows from the second to the first. Formulae merge — `self`’s formulae take priority on conflict.

`-` removes exact matching rows. Whole-row set difference. If a row appears in the right operand, it’s removed from the left. Both operators raise `ArgumentError` when dimensions differ.

```
jan_sales + feb_sales      # more rows, same dimensions
all_sales - returned_items # fewer rows, same dimensions
```

0.5.0 (2026-04-16): Intersection, union, symmetric difference

The remaining row-axis set operators, completing the set algebra.

`&` returns rows present in both operands. `|` returns all rows deduplicated (implemented via `Array#|`). `^` returns rows in one operand but not both — the symmetric difference.

```

sales & promotions      # rows in both
sales | new_arrivals    # all rows, no duplicates
sales ^ competitor      # rows unique to each side

```

All three require matching dimensions, raise `ArgumentError` otherwise. Formulae carry through from self, merge from other, self wins on conflict. `|` and `^` merge formulae from both sides.

0.6.0 (2026-05-11): Equality, pattern-match, subset, and superset

Equality, pattern-match, and subset/superset operators, multiset-theoretic on rows where row-content matters. Two Namos with the same rows in different orders are equal; two Namos where one's rows are contained in the other are in a subset relation regardless of how either was ordered. Duplicate rows count — `[{a: 1}, {a: 1}]` is not equal to `[{a: 1}]`. This follows from Namo's identity as a multidimensional database rather than a sequence: the comparison operators treat the rows as a multiset, so their order does not affect equality, while row count does. Row order is preserved by the row-sequence operations (`to_h/values`, `each`, `first/last/take/drop`, `+`); it is the comparison operators specifically that disregard it.

The equality hierarchy mirrors Ruby's standard convention, extended with `===` for pattern-match dispatch:

```

a.equal?(b)    # same object (inherited from BasicObject, not overridden)
a.eql?(b)      # same class + multiset-equal data + same formula names
a == b         # multiset-equal data, any class, formula names ignored
a === b        # same dimensions + same formula names, any class, data ignored

```

`eql?` requires class match because subclassing carries included modules (`TradingAnalysis < Namo` includes `Indicators`, `Scoring`); two instances of the same subclass with the same data and formulae are interchangeable as analyses, while a subclass and a bare Namo with the same data are not. `==` ignores class and formulae — two Namos are equal if they hold the same data, mirroring Ruby's `1 == 1.0 / 1.eql?(1.0)` convention.

`===` answers a structurally different question: does this candidate fit this analytical pattern? Same dimensions, same formula names, regardless of the rows. This is what case statements need — pattern dispatch on analytical shape, not on data identity:

```
template = Namo.new([{:x: 0}])
```

```

case Namo.new([{:x: 5}, {:x: 6}])
when template then :matched
else :not_matched
end
# => :matched

```

`===` returns false (rather than raising) for non-Namo operands, since case statements require it to be safe to call on any value. Without a Namo-specific `===`, Ruby's default would delegate to `==`, which would dispatch case statements on multiset row equality — structurally misleading when users want analytical-type dispatch.

```

namo_a = Namo.new([{:symbol: 'BHP', close: 42.5}, {:symbol: 'RIO', close: 118.3}])
namo_b = Namo.new([{:symbol: 'RIO', close: 118.3}, {:symbol: 'BHP', close: 42.5}])

```

```

namo_a == namo_b      # true - same rows, different order
namo_a.eql?(namo_b)   # true - same class, same rows, no formulae either side
namo_a.equal?(namo_b) # false - different objects

```

`hash` is content-based and consistent with `eql?` — computed from the canonical form of `@data` (rows sorted lexicographically), the class, and the formula names. Two Namos that are `eql?` produce the same hash, making Namos usable as Hash keys and Set members.

Subset and superset follow `stdlib Set`'s precedent, generalised to multisets:

```
a < b    # a's rows are a strict multiset subset of b's rows
```

```

a <= b    # a's rows are a multiset subset of b's rows
a > b     # a's rows are a strict multiset superset of b's rows
a >= b    # a's rows are a multiset superset of b's rows

```

These pair with the set operators algebraically: $a \& b == a$ iff $a \leq b$; $a \mid b == b$ iff $a \leq b$; $a - b == \emptyset$ iff $a \leq b$. Dimension mismatch raises `ArgumentError`; non-Namo operand raises `TypeError`.

The error message format for dimension mismatch was standardised to "dimensions don't match: X vs Y" across all binary operators (+, -, &, |, ^, <, <=, >, >=), retrofitted into the set operators from 0.4.0–0.5.0.

Deliberately not included: `<=>` and `Comparable` (subset/superset is a partial order, not total — two Namos can be incomparable, and `stdlib Set` omits `<=>` for the same reason); Row-level public operators; block forms (relaxed-matching variants land in 0.14.0 alongside block forms for set and composition operators).

0.7.0 (2026-05-20): Derived-dimension surfacing, values, live views

Three things land together: values as the third inspection aspect alongside dimensions and coordinates; derived dimensions (formula names) surfacing through the inspection aspects so data and derived are queried uniformly; and explicit accessors for the data/derived split (`data_dimensions`, `derived_dimensions`). All three accessors are live — they recompute from current state on every call, with no memoisation.

The shape of the API is plain Ruby. `dimensions` returns an `Array`, `coordinates` and `values` return `Hash` (eager) or `Array` (single column, lazy), depending on how they're called. No aspect classes, no subclasses of `Array/Hash`. Subclassing of `Namo` itself (`class Sales < Namu`) and `Namu#===` from 0.6.0 already cover case-statement dispatch on analytical shape, so the aspect-class layer is not needed.

values Extract per-dimension sequences, preserving duplicates and row order.

```

namo = Namu.new([
  {symbol: 'BHP', close: 42.5},
  {symbol: 'RIO', close: 118.3},
  {symbol: 'BHP', close: 43.1},
])

namo.values(:symbol)      # => ['BHP', 'RIO', 'BHP']
namo.coordinates(:symbol) # => ['BHP', 'RIO']

```

`coordinates` answers “what exists along this axis?” — the unique axis labels. `values` answers “what does this column actually contain?” — the raw data, preserving count and order. The difference is analogous to `DISTINCT` vs a bare `SELECT` on a column.

Both accept positional dimension arguments:

```

namo.values                # => full Hash {dim => sequence}
namo.values(:symbol)       # => Array – one column, lazily computed
namo.values(:symbol, :close) # => subset Hash with just those columns
namo.values(:unknown)      # => [nil, nil, ...] – one nil per row
namo.values(:symbol, :unknown) # => {symbol: [...], unknown: [nil, nil, ...]}

```

With no args, `values` returns a `Hash` keyed by every dimension in the queryable namespace. With one arg it returns just that column as an `Array`, computing only that column (lazy). With multiple args it returns a `Hash` containing just the requested columns, again computing only those. Unknown dimensions propagate `nil` naturally — `row[:unknown]` is `nil` for each row, so the column is an `Array` of `nil`s — matching the convention used by `Row#[]` and `Namu#[]` selection. `coordinates` mirrors the same shape; an unknown dimension becomes `[nil]` after `uniq`ing the per-row `nil`s.

Asymmetric returns (`Array` for one arg, `Hash` for none or many) follow the natural reading: `values(:symbol)` is “the values of `:symbol`,” singular, so an `Array`. Ruby has precedent for arg-shape-driven returns (`arr[2]` vs `arr[2..3]`); the principle is that the most useful shape for each calling pattern wins over uniformity.

The implementation is straightforward and lives directly in `Namo`:

```
def values(*dims)
  if dims.empty?
    dimensions.each_with_object({}){|dim, hash| hash[dim] = values_for(dim)}
  elsif dims.length == 1
    dim = dims.first
    return nil unless dimensions.include?(dim)
    values_for(dim)
  else
    dims.each_with_object({}){|dim, hash| hash[dim] = values_for(dim) if dimensions.include?(dim)}
  end
end

def coordinates(*dims)
  if dims.empty?
    values.transform_values(&:uniq)
  elsif dims.length == 1
    values(dims.first)&.uniq
  else
    dims.each_with_object({}){|dim, hash| hash[dim] = values(dim)&.uniq if dimensions.include?(dim)}
  end
end
```

`coordinates` is literally `values` with `.uniq` applied per column — a single source of truth for per-dimension column extraction. `coordinates(dim) == values(dim).uniq` holds across the queryable namespace, by construction.

Queryable namespace: data and derived dimensions together `dimensions`, `values`, and `coordinates` all cover the *queryable namespace* — the union of data dimensions (keys of `@data.first`) and derived dimensions (keys of `@formulae`). From the user’s perspective, data and derived are equivalent: anywhere you can name a data dimension, you can name a formula.

```
namo = Namu.new([{:price: 10.0, quantity: 100}, {:price: 25.0, quantity: 60}])
namo[:revenue] = proc{|r| r[:price] * r[:quantity]}
```

```
namo.dimensions      # => [:price, :quantity, :revenue]
namo.values(:revenue) # => [1000.0, 1500.0]
namo.coordinates(:revenue) # => [1000.0, 1500.0]
namo.to_h            # => {price: [...], quantity: [...], revenue: [...]}
```

Asking for `values(:revenue)` evaluates the formula across every row and returns the resulting column. The derived dimension behaves identically to a data dimension from the inspection vocabulary’s point of view. This unification matches the 0.1.0 design principle that formulae are first-class dimensions, not a separate computed-column concept; 0.7.0 finishes wiring that principle through the inspection methods.

Data and derived as explicit accessors For the cases where the distinction does matter — set operators need to compare row schemas, projection through `[]` operates on data columns, and code introspecting a `Namo` may want to know which names are data vs derived — the split is exposed as parallel accessors:

```
namo.data_dimensions  # => [:price, :quantity]  – keys of the first row
namo.derived_dimensions # => [:revenue]         – keys of @formulae
namo.dimensions       # => [:price, :quantity, :revenue] – the union, queryable namespace
```

The set operators (`+`, `-`, `&`, `|`, `^`) use `data_dimensions` for their matching check internally, preserving the 0.6.0 semantics that two `Namos` with the same data layout can be combined regardless of which formulae

either has registered.

to_h to_h is the Ruby-conventional alias for values with no arguments:

```
def to_h
  values
end
```

Returns the full columnar Hash. values is the primitive name; to_h reads naturally at coercion sites (hash = namo.to_h; hash.keys).

This is purely about output. Internal storage stays row-oriented (@data as Array<Hash>); columnar storage as a performance option is a 3.x concern. The public to_h/values shape is stable from 0.7.0.

The pairing of values with proc-based selection (0.8.0) is about user workflow: if you're writing namo[pe: ->(v){ v && v < 15 }], you probably also want namo.values(:pe) first to inspect the range and spot nils before writing the predicate.

Live views, no memoisation Every call to dimensions, data_dimensions, derived_dimensions, values, coordinates, or to_h recomputes from the Namos current state. There is no caching. Mutating @data or @formulae (e.g. via namo[:new_formula] = proc{...}) is reflected immediately on the next inspection call.

```
namo.derived_dimensions # => []
namo[:revenue] = proc{|r| r[:price] * r[:quantity]}
namo.derived_dimensions # => [:revenue]
namo.values(:revenue)   # => [1000.0, 1500.0]
```

The performance cost compared to a memoised implementation is real but unmeasured at typical sizes (hundreds to low thousands of rows). Single-column laziness — values(:dim) computing only the requested column — covers the case where it matters most: a million-row Namos with expensive derived dimensions, where materialising the full Hash would be wasteful. Wholesale aspect caching (freeze-aware materialisation) lands in 2.x as an opt-in, transparent optimisation — see 2.x's caching subsection. It is deliberately not in 1.x: the whole 1.x line is pure-live, recomputing derived views from current state on every access, so that liveness is unconditional and there is no staleness to reason about. Caching is a 2.x performance concern gated on freeze, never a behaviour change.

Composition through plain types Because dimensions, data_dimensions, and derived_dimensions return plain Array, and values(:dim) and coordinates(:dim) return plain Array, the full vocabulary of Ruby's Array operators applies without Namos defining anything new:

```
a.dimensions == b.dimensions           # exact match on queryable namespace
a.data_dimensions == b.data_dimensions  # exact match on data layout
a.derived_dimensions == b.derived_dimensions # exact match on formula names
a.coordinates(:symbol) == b.coordinates(:symbol) # set equality on a dimension
a.values(:close) == b.values(:close)      # sequence equality on a dimension
a.values(:close).sum                    # any Array method
namos.sort_by{|n| n.values(:date).first}  # sort Namos by aspect
[:price, :quantity].all?{|d| a.dimensions.include?(d)} # subset-style check
```

This is the answer to “where does ordering live?” — at the aspect level, where projections to plain Arrays inherit <=> and == from Ruby's built-ins. No Namos#<=>, no Namos-level total order. Just plain return types.

Case-statement dispatch: subclassing and Namos#=== Case-statement dispatch on analytical shape has two paths, both already present without 0.7.0 adding any new === infrastructure:

```

# Subclassing – the cleanest path when you have named shapes
class Sales < Namu; end
class Inventory < Namu; end

case incoming
when Sales      then publish_sales_dashboard(incoming)
when Inventory then publish_inventory_dashboard(incoming)
end

# Namu#=== – for ad-hoc shape templates without a subclass
ohlc = Namu.new([{:symbol: nil, date: nil, open: nil, high: nil, low: nil, close: nil, volume: nil}])
case incoming
when ohlc then process_ohlc(incoming)
end

```

Class#=== dispatches on `is_a?` (stock Ruby), so subclasses of `Namu` flow through case statements directly, inheriting case dispatch alongside their domain methods. `Namu#===` (from 0.6.0) matches on full analytical shape — same dimensions, same formula names — for templates that don't justify a subclass.

Aspect-level === (the originally-planned `ohlc.dimensions === incoming` template-match) is not part of 0.7.0. The case-dispatch use it would have served is covered by subclassing for known shapes and `Namu#===` for ad-hoc ones; the data-only and derived-only fine-grained variants don't have a concrete use case beyond what `==` on the relevant accessor already gives (`a.data_dimensions == b.data_dimensions`, etc.). If a case-dispatch need on the finer split materialises later, a small `Matcher` returned by a factory method on `Namu` can serve it without bringing back the aspect-class hierarchy.

Why Array storage, not Set `values` reinforces the decision to keep `@data` as `Array<Hash>` rather than `Set<Hash>`. `values` needs ordering and duplicates — both things `Set` throws away. If the internal store were a `Set`, `values` would lose row order and couldn't contain duplicate rows. `to_a` would need to reconstruct an order that no longer exists internally. `to_h` (columnar) would have the same problem — the column arrays need a consistent row ordering so that `values(:symbol)[i]` and `values(:close)[i]` correspond to the same row. `Set` buys fast membership testing and automatic deduplication, but `Namu` doesn't need either of those as primitives. The set operations (`&`, `|`, `^`) work on `Array` storage just fine.

0.8.0 (2026-05-21): Proc-based and regex-based selection

Two ways to extend `[]` selection beyond exact values, arrays, and ranges: procs for arbitrary predicates, regexes for string pattern matching. Both are single-branch additions to existing selection logic, paired here because they share the same dispatch site (`Row#match?`).

Proc-based selection `[]` accepts procs as selection predicates on any dimension. The proc receives the dimension value (or `nil` for a missing or `nil`-valued dimension) and decides — `truthy` result selects the row.

```

namu[pe: ->(v){v && v < 15}]
namu[price: ->(v){v > 10.0}, symbol: ->(v){v != 'TEST'}]

```

Implementation is a single `when Proc` branch in `Row#match?` calling `coordinate.call(self[dimension])`. The match is on `Proc` specifically — not duck-typed via `respond_to?(:call)` — so lambdas, procs, and `Symbol#to_proc` results all flow through, but methods and other callables don't. Exceptions from the predicate propagate; no rescue clause.

This enables multi-factor screening in one expression:

```

namu[pe: ->(v){v && v < 15}, price_to_book: ->(v){v && v < 1.5}]

```

Proc-based selection composes with contraction and projection in a single `[]` call, and works on formula-defined dimensions:

```
namo[:revenue] = proc{|r| r[:price] * r[:quantity]}
namo[revenue: ->(v){v >= 1500.0}]
```

Regex-based selection [] accepts regexes as selection predicates on any dimension. The dimension value is coerced with `to_s` before matching, so regexes work against strings, symbols, integers, floats, dates, and anything else with a sensible string form.

```
namo[symbol: /^BH/]          # symbols starting with BH
namo[symbol: /gold/i]        # case-insensitive match
namo[sector: /mining|resources/i] # alternatives
namo[symbol: /^BH/, sector: 'Energy'] # regex + exact, composable
```

Implementation is a single `when Regexp` branch in `Row#match?`:

```
when Regexp
  coordinate.match?(self[dimension].to_s)
```

Same weight as adding `proc` support — one additional `when` branch in the same case statement.

The `to_s` coercion has predictable behaviour across types:

value	.to_s	matches //	matches /. /
nil	""	yes	no
42	"42"	yes	yes
10.0	"10.0"	yes	yes
:priority	"priority"	yes	yes
Date.new(...)	"2026-05-21"	yes	yes

Regex is more ergonomic than the equivalent `proc` for string pattern matching:

```
# Regex
namo[symbol: /^BH/]

# Equivalent proc
namo[symbol: ->(v){v.to_s =~ /^BH/}]
```

The regex form is shorter, more declarative, and immediately legible. It doesn't replace procs — procs handle arbitrary logic — but for pattern matching on string-valued (or string-coercible) dimensions it's the natural tool.

Regex composes with all other selection types in the same [] call: exact values, arrays, ranges, procs, projection, and contraction.

0.9.0 (2026-05-21): Composition operators (*, **, /)

The dimensional composition algebra. Three operators that extend `Namo` from the same-dimensions algebra (set operators in 0.4.0–0.5.0, comparison operators in 0.6.0) to combining and decomposing `Namos` with different dimensions.

*** (equi-join on shared dimensions)** Pairs rows where coordinates match on every shared data dimension. Inner-join semantics — unmatched rows from both sides are dropped. Output dimensions are `self.data_dimensions` followed by other's exclusive dimensions; output multiplicity is the product of input multiplicities on each matching key.

```
ohlcv * fundamentals # joins on shared :symbol
```

Requires at least one shared data dimension. No overlap raises `ArgumentError` — silently falling through to a Cartesian product would turn a logic error into a large pile of nonsense rows. Formulae merge from both sides; self wins on conflict.

**** (Cartesian product)** Every row from the left paired with every row from the right. Output has `self.data.length * other.data.length` rows; output dimensions are `self.data_dimensions + other.data_dimensions`.

products ** quarters

Requires **no** shared data dimensions — the precondition is the mirror image of `*`. Any overlap raises `ArgumentError`. The visual relationship is deliberate: `*` is the filtered version, `**` is the explosive version — more sigil, more output. Formulae merge from both sides; self wins on conflict.

/ (decomposition) Removes from self the dimensions that are also in other (the intersection), then dedupes the projected rows. The inverse of `*` and `**`.

combined / fundamentals # removes shared dimensions, keeps everything else

No precondition — `/` is total on $\text{Namo} \times \text{Namo}$. When the operands share no dimensions, the intersection is empty and `self / other` returns a `Namo` equal to self. The asymmetric strictness — `*` and `**` raise, `/` is loose — reflects a structural distinction: `*` and `**` are *combining* operators that need a specific relationship between operands to produce a meaningful result, while `/` is a *projecting* operator where “project away nothing” has a natural answer (“return the original”).

The looseness earns `/` algebraic properties a strict version would lose: identity test (`c / b == c` iff they share no dimensions), idempotence (`(c / b) / b == c / b`), and pipeline composition (a step that applies `/` separator can run over any `Namo` without special-casing applicability).

Round-trip identity:

- `(a ** b) / b == a` exactly.
- `(a * b) / b == a[-:shared]` — the dimensions shared with `b` are lost on decomposition.

The asymmetry between the two round-trip cases is real: `/` operates only on the two values it receives and cannot distinguish “shared dimension that belonged to both” from “exclusive dimension that belonged only to the right”. Removing the intersection is the only rule expressible from the operands alone.

0.10.0 (2026-05-28): Row comparison

Extends 0.6.0’s comparison work one level down. 0.6.0 settled comparison at the `Namo` level — `==`, `eq?`, `hash`, `===`, and the subset/superset operators — but left `Row` without value semantics. The omission was defensible at the time: `Row` read as implementation detail behind `Namo`’s algebra. Applied use showed otherwise. Rows leak through each, get reached for directly in interactive sessions, and become prerequisites for the dedup, hash-keying, and Row-against-Row equality that 0.11.0’s `Enumerable` coherence pass needs. This release closes that gap: `Row` gets `==`, `eq?`, and `hash` matching its role as a hash-shaped value.

The pattern of 0.10.0 revisiting 0.6.0 is worth flagging because 0.11.0 revisits 0.2.0 in the same way — the substrate beneath an earlier algebra needs to catch up to applied use. The “Notes on these two releases” at the end of 0.11.0 expands on the pattern.

== Data equality. Two Rows are equal if their underlying `@row` hashes are equal. Formulae are not part of equality — they’re attached by the surrounding `Namo`, not properties of the row.

```
def ==(other)
  other.is_a?(Row) && @row == other.to_h
end
```

This mirrors `Namo#==` one level down. 0.6.0's `Namo#==` ignores class and formulae and compares data; `Row#==` follows the same rule. Two Rows with identical `@row` data are equal regardless of which `Namo` they came from.

eql? Mirrors `Row#==` in shape — an `is_a?(Row)` gate followed by a hash comparison. Unlike `Namo#eql?`, there's no class-identity gate (no `self.class == other.class`); Rows don't have a subclass hierarchy with included modules, so there's no class story to enforce.

```
def eql?(other)
  other.is_a?(Row) && @row.eql?(other.to_h)
end
```

The difference from `Row#==` is that `eql?` uses `Hash#eql?` for the underlying comparison rather than `Hash#==`, matching Ruby's convention (`1 == 1.0` is true but `1.eql?(1.0)` is false; Hash comparison follows from its keys' and values' comparison). The Row-level consequence is that `Row.new({n: 1}, {})` `==` `Row.new({n: 1.0}, {})` is true but `.eql?` between the same two is false.

hash Consistent with `eql?` — Rows that are `eql?` produce the same hash, making them usable as Hash keys and Set members.

```
def hash
  @row.hash
end
```

Why these three, not the full Nammo stack `Namo` gets `==`, `eql?`, `hash`, `===`, `<`, `<=`, `>`, `>=` because a `Namo` is a collection of rows with set-theoretic relationships. A `Row` is a record — a single hash-shaped value. The set-theoretic operators don't translate: a `Row` isn't a "subset" of another `Row` in any meaningful sense, and `===` for pattern-match dispatch doesn't apply to row-level values the way it applies to analytical shapes. The three that translate are the value-semantics trio: `==`, `eql?`, `hash`. Those are what a hash-shaped value needs to participate in Ruby's collection machinery correctly.

0.11.0 (2026-05-31): Enumerable methods return Namos

Extends 0.2.0's `Enumerable` inclusion. 0.2.0 made `Namo` `Enumerable` but left Ruby's default return types in place — `select`, `reject`, and `friends` produced Arrays, breaking the analytical chain. Applied use made the friction visible: every interactive session that selects rows from a `Namo` and tries to continue working with the result hit a `NoMethodError` for selection or projection on the Array. 0.11.0 specialises the subset-returning `Enumerable` methods to wrap in `Namo`. The Row-equality groundwork for `uniq` and `partition` came in from 0.10.0.

These are sequence-view operations: `select`, `reject`, `sort_by`, `first(n)`, `last(n)`, `take(n)`, `drop(n)`, `take_while`, `drop_while`, `uniq`, and `partition` all care about row order and produce ordered subsets. They sit alongside the set-view operators (`==`, `<`, `&`, `|`, etc.) from 0.4.0–0.6.0. `Namo`'s dual nature — set when membership is what matters, sequence when order is what matters — is now realised across both families: set operators ignore order and produce set-correct results; `Enumerable` methods respect order and produce ordered results. The same `Namo` supports both views.

```
# Before (0.2.0–0.10.0)
filtered = namo.select{|row| row[:close] > 40.0}
filtered.class      # => Array
filtered[:symbol: 'BHP'] # => NoMethodError

# After (0.11.0)
filtered = namo.select{|row| row[:close] > 40.0}
filtered.class      # => Nammo
filtered[:symbol: 'BHP'] # works – selection, projection, formulae, everything
```

Scope Methods that return Namos:

- `select`, `reject` — predicate-based subset, formulae carry through. `select`'s Enumerable aliases `filter` and `find_all` are aliased to the override, so they return Namos too — the only overridden method that has aliases to keep in step.
- `sort_by` — reordered Namo, formulae carry through.
- `first(n)`, `last(n)` — leading and trailing subsets. Without an argument, return a single Row (or nil for empty); with an argument, return a Namo.
- `take(n)`, `drop(n)` — leading subset and its complement.
- `take_while`, `drop_while` — predicate-based leading subset and its complement.
- `uniq` — dedupe rows on full-row equality (`Row#==/ eql?` from 0.10.0). With a block, dedupe on the block's return value, following Enumerable#`uniq`'s convention.
- `partition` — returns `[Namo, Namo]` — matches and non-matches.

Methods that did not change: `map`, `flat_map` (transformed values, possibly not row-shaped), `reduce`, `sum`, `min_by`, `max_by`, `count` (scalars), and `each` (already returns an Enumerator or yields Rows).

`group_by` is deliberately excluded, and not merely deferred — it is *structurally blocked*. Every method above returns a Namo (or `[Namo, Namo]`), and Namo exists. `group_by` returns a keyed set of sub-Namos, and the right type for that is `Namo::Collection`, which doesn't exist until 0.17.0. It lands at 0.18.0.

Implementation pattern Subset-returning methods construct via `self.class.new` so subclass type is preserved; formulae carry through, duped to avoid shared mutation.

```
def select(&block)
  self.class.new(@data.select{|row| block.call(Row.new(row, @formulae))}, formulae: @formulae.dup)
end

def first(n = nil)
  if n
    self.class.new(@data.first(n), formulae: @formulae.dup)
  else
    @data.first ? Row.new(@data.first, @formulae) : nil
  end
end

def uniq(&block)
  rows = block ? @data.uniq{|row| block.call(Row.new(row, @formulae))} : @data.uniq
  self.class.new(rows, formulae: @formulae.dup)
end

def partition(&block)
  matches, non_matches = @data.partition{|row| block.call(Row.new(row, @formulae))}
  [
    self.class.new(matches, formulae: @formulae.dup),
    self.class.new(non_matches, formulae: @formulae.dup),
  ]
end
```

The no-block `uniq` dedupes raw `@data` hashes via `Array#uniq`, which uses `eql?/hash` — exactly what `Row#eql?/Row#hash` delegate to — so it matches Row equality while avoiding Row allocations. The practical consequence is numeric type strictness: `Namo.new({n: 1}, {n: 1.0}).uniq` keeps both rows, matching `Row#eql?`.

As of 0.11.1, these methods (and `each`) live in a `Namo::Enumerable` module included into Namo, rather than inline on the class — a behaviour-preserving reorganisation. The module include `::Enumerables` itself

and sits above `stdlib Enumerable` in Namo's ancestor chain, so the overrides win while `map/reduce/etc.` fall through unchanged.

Subclass considerations Same trap as the operators — `self.class.new(...)` constructs an instance of the subclass, firing its `initialize` side effects. The `if name guard` pattern (introduced in 0.12.0 alongside the `name: attribute`) is the documented convention for subclasses with side effects in `initialize`.

Backward compatibility A breaking change for code expecting `Array` returns from these methods. In 0.x, breaking changes are allowed; the release note: “Subset `Enumerable` methods now return `Namos`. If your code expected `Arrays`, call `.entries` or `.to_a` on the result.”

Performance note `last(n)` goes straight to `@data.last(n)`, the efficient path. No fall-through to `Enumerable`'s materialise-then-slice behaviour.

Notes on these two releases 0.10.0 and 0.11.0 are a single conceptual move. 0.10.0 extends 0.6.0's comparison work to `Row`; 0.11.0 extends 0.2.0's `Enumerable` inclusion to make subset-returning methods produce `Namos`. They pair because 0.11.0's `uniq` and `partition` need `Row` equality to work correctly, and 0.10.0 puts that in place.

The pattern of revisiting earlier releases is worth flagging because it recurs. Namo's earlier releases closed algebras cleanly at one conceptual level — `Enumerable` at 0.2.0, the set algebra at 0.4.0–0.5.0, comparison at 0.6.0, composition at 0.9.0. Each was a coherence statement: a complete, deliberately-bounded piece of algebra. Theoretical design closes algebras at their natural level; the substrate beneath the algebra (return types, value semantics on the underlying objects) tends to get left at Ruby defaults because the design effort goes into the structural completeness above.

Applied use stresses different parts. Interactive sessions reveal that `select` returning `Array` breaks chaining. Wanting to dedupe rows reveals that `Row` never got the value semantics the Namo level got. The substrate beneath the algebra needs to catch up to applied use. This is healthy: the earlier releases are sound — the revisits are extensions, not corrections. If applied use were revealing genuine design errors that would be different; instead it reveals where deliberate scope limits in earlier releases were tighter than the applied surface needs. Future releases may follow the same pattern.

0.12.0 (2026-06-01): Constructor widening — keyword `data` and `name`:

One coherent edit to `initialize`: the constructor grew two optional keyword arguments. `Data` can now be passed by the `data:` keyword as well as positionally, and a `Namo` can carry a `name`. Both are additive — every existing call site (`Namo.new([ {...} ])`, `Namo.new([ {...} ], formulae: {})`, the no-arg and formulae-only forms, and the operators' internal `self.class.new(rows, formulae: ...)`) is unaffected.

```
# Before (through 0.11.x)
def initialize(data = [], formulae: {})

# 0.12.0 – widened
def initialize(positional_data = nil, data: [], formulae: {}, name: nil)
  @data = positional_data || data
  @formulae = formulae
  @name = name
end
```

Positional data wins when both positional and keyword `data:` are present. The positional default changed from `[]` to `nil` so it acts as a “not supplied” sentinel that lets `|| data` fall through to the keyword path; an explicit `Namo.new([])` still yields `@data == []` because the truthy empty array short-circuits.

```
Namo.new([{:x: 1}])      # positional – unchanged
```

```
Namo.new(data: [{x: 1}])      # keyword – new
Namo.new([{:x: 1}], data: [{x: 2}])  # positional wins → @data == [{x: 1}]
```

name: is the load-bearing half — the driver is `Namo::Collection` (0.17.0), where members identify themselves so the collection can find them by name, replace them on re-add, and label them in summaries. `Namo` gains `attr_accessor :name`; `name=` gives post-construction mutation. Operator-derived `Namos` (`+`, `*`, `select`, ...) construct without `name`, so their `@name` is `nil` — a derived object is not the original, and giving it the parent's name would mislead.

That `nil`-on-derivation behaviour is what makes the subclass guard convention work: subclasses guard initialize side effects with `return unless name`, so operator-derived instances skip them and only explicitly-named constructions fire them. `super` with no parentheses forwards every argument unchanged.

```
class TradingAnalysis < Namu
  def initialize(positional_data = nil, data: [], formulae: {}, name: nil)
    super
    return unless name
    register_indicators
  end
end
```

This is the answer to the “operator return type” open question for subclasses with side effects in `initialize` — they need not override every operator to stop the result of `*` or `select` from re-running construction; they guard on `name`. The convention is documented in the README and applied throughout `Namo`'s own subclasses.

These two changes shipped together because they are the same modification: the constructor's signature growing optional keyword parameters. Polymorphic `[]=` (0.13.0) was deliberately not bundled — it changes assignment dispatch, a different method and a different concern. The seam is “constructor-signature changes together; assignment-dispatch changes separately.”

0.13.0 (2026-06-01): Polymorphic `[]=`

`[]=` dispatches on the type of the value assigned. A `Proc` registers a formula; anything else broadcasts the value to every row. The two branches are mirror images that together enforce **exclusive storage** — a name is either a data dimension or a derived dimension, never both.

```
def []=(name, value)
  case value
  when Proc
    @data.each{|row| row.delete(name)} if @data.first&.key?(name)
    @formulae[name] = value
  else
    @formulae.delete(name)
    @data.each{|row| row[name] = value}
  end
end
```

The `Proc` branch clears any data column of that name before registering the formula; the `else` branch deletes any formula of that name before broadcasting. Last write wins, and there is no shadowing. The `Proc`-branch clear is guarded (`if @data.first&.key?(name)`) so it walks the rows only when there's actually a data column to clear; the `&.` also handles the empty-data case where `@data.first` is `nil`. The `else` branch's `@formulae.delete(name)` and broadcast are unconditional — `Hash#delete` on an absent key is a cheap no-op, and the broadcast is the operation's actual work. The match is on `Proc` specifically (`Proc === value`), consistent with `Row#match?`'s `proc-selection` branch from 0.8.0, so an array is not a `proc` and `namo[:weights] = [1, 2, 3]` broadcasts the array as each row's value.

Exclusivity is self-enforcing because the inspection vocabulary is derived live. `data_dimensions` reads the

keys of the first row and `derived_dimensions` reads the keys of `@formulae`, so clearing the data column is what removes a name from `data_dimensions`, and deleting the formula is what removes it from `derived_dimensions`. There's no separate bookkeeping to keep in sync. A name assigned a scalar shows up in `data_dimensions`; a name assigned a proc shows up in `derived_dimensions`; never both, so it appears in `dimensions` (their concatenation) exactly once.

Why polymorphic, not a separate method The alternative — `[]=` for formulae plus a separate `broadcast/set_all` for scalars — was considered and rejected. `[]` already dispatches polymorphically over exact values, arrays, ranges, procs, and regexes (0.8.0); `[]=` follows the same convention rather than introduce a parallel naming scheme. The exclusivity rule then lives at the one place assignment happens, instead of being pushed onto the user, who could otherwise leave a formula shadowed by a data column of the same name (or vice versa) — stale state that, with the live-derived inspection methods, would surface as a name listed in both `data_dimensions` and `derived_dimensions`. And it reads naturally: `namo[:status] = 'active'` is “set status to active across this Namu,” which is exactly what it does.

This shipped separately from the 0.12.0 constructor widening by design: that release changed the constructor's signature; this one changes assignment dispatch — a different method and a different concern. The seam is “constructor-signature changes together; assignment-dispatch changes separately.”

0.14.0 (2026-06-08): Blocks on composition operators (*, **)

`*` and `**` take an optional block that refines which rows pair. Without a block, both behave exactly as they did in 0.9.0 — `*` pairs every shared-dimension match, `**` pairs every row with every row. With a block, the operator hands it the current left row and a `Namu` of candidate right rows, and the block returns the subset to pair.

```
block.call(row, candidates) # => Namu
```

`row` is the `Row` for the current left row, carrying `self`'s formulae (the same object each yields), so `row[:date]` and any `self` formula resolve inside the block. `candidates` is a `Namu` of right rows carrying other's formulae — for `*`, the rows already matched on the shared data dimensions; for `**`, all of other's rows, with no pre-filter — so the block can select on other's derived dimensions as readily as its data. The block returns a `Namu` of the rows to pair, each merged into the left row. It is a selector, not a reducer: zero, one, or many rows are all valid returns. An empty returned `Namu` pairs nothing, dropping that left row — inner-join semantics preserved.

The canonical use is a matching rule plain `*` cannot express because it pairs every match: match each daily price to a single quarterly report — the most recent one dated on or before it.

```
prices.*(quarterly) do |row, candidates|
  candidates[quarter_end: ->(qe){qe <= row[:date]}].sort_by{|f| f[:quarter_end]}.last(1)
end
```

`**`'s block is the parallel for the no-shared-dimensions case — a conditional product, pairing each order with only the shipping tiers that can carry it:

```
orders.**(tiers) do |row, candidates|
  candidates[max_weight: ->(w){w >= row[:weight]}]
end
```

The block decides only which rows pair; it does not touch the result's formulae. Those are set once by the operator, `other.formulae.merge(@formulae)`, identical to the no-block path — `candidates` carries other's formulae only so block selection on other's derived dimensions resolves, read-only and local to the block, and the returned `Namu` contributes row data only. So formula handling is byte-identical between the block and no-block forms. A derived dimension of `other` (a formula, not a stored key) does not become stored data on merge; it reappears on the result because the result carries other's formulae, resolving to the same value. The operators' preconditions are unchanged and additive: `*` still requires a shared dimension and `**` still requires disjoint dimensions, block or no block. A `**` block that returns its candidates unchanged reproduces

the no-block product, and one that matches on shared dimensions by hand reproduces no-block * — * is ** with the shared-dimension match applied first.

The governing principle A block form is warranted exactly when the operation gives consideration to a dimension in isolation. The composition operators do — * singles out the shared dimensions as the join axis and the block reasons about particular dimensions' values; ** gives no such consideration itself, and the block is where the user supplies it. The set operators (+, -, &, |, ^) do not: they act on the whole row as an indivisible value (whole-row equality via eql?/hash), singling out no dimension, so there is nothing for a block to refine or supply. Decomposition (/) considers dimensions collectively — a set of names operated on wholesale — never a dimension in isolation, so it too takes no block.

This ties to orthogonality: where a conditional operation can be expressed by composing existing orthogonal operations, it should be, rather than by adding a block to an operator. The anti-join a set-operator block would have served is one such case — today[symbol: ->(s){!excluded.include?(s)}] (with excluded a Set for an O(today + exclusions) lookup) expresses it using only shipped features, and is at least as efficient as a block form would be. A block earns its place only where no such composition exists — precisely the composition operators, where the dimension-in-isolation matching is underdetermined and cannot be assembled from existing parts. The comparison operators are excluded on the same ground: a keyed comparison is already today[:symbol] == yesterday[:symbol], a comparison of projections, so no block is needed; and eql? would not take one regardless, its job being the strictest equality.

Summary

The set operators (+, -, &, |, ^), the comparison operators (==, ===, eql?, <, <=, >, >=), and the composition operators (*, **, /) — * and ** taking optional blocks for custom match refinement — together with selection (exact, array, range, proc, regex), projection, contraction, formulae, polymorphic assignment via []= (proc registers a formula, scalar broadcasts to every row, exclusive storage either way), the full inspection vocabulary (dimensions, data_dimensions, derived_dimensions, coordinates, values, to_h), Row value semantics (==, eql?, hash), the subset-returning Enumerable methods (select, reject, sort_by, first, last, take, drop, take_while, drop_while, uniq, partition) returning Namos, and a constructor that takes data positionally or by keyword and carries an optional name:, give Namo a complete vocabulary for working with a single dataset, combining datasets that share the same dimensions, and combining or decomposing datasets with different dimensions, with Rows that behave correctly as Ruby values and analytical chains that stay closed through filtering and ordering. The next phase (0.15.0) adds two-arity formulae for cross-row computation, then proceeds through richer formulae and Namo::Collection.

0.15.0: Two-arity formulae

Row gains a reference to its parent Namo. Procs with arity 2 receive (row, namo), enabling cross-row computation:

```
e[:sma_20] = proc do |row, namo|
  window = namo[symbol: row[:symbol], date: ..row[:date]].last(20)
  window.sum{|r| r[:close]} / window.length.to_f
end
```

The each method passes self (the Namo) to Row:

```
def each(&block)
  return enum_for(:each) unless block_given?
  @data.each{|row_data| block.call(Row.new(row_data, @formulae, self))}
end
```

Row's constructor gains an optional Namo reference:

```
class Row
  def initialize(row, formulae, namo = nil)
```

```

    @row = row
    @formulae = formulae
    @namo = namo
  end
end

```

The `namo` parameter defaults to `nil` for backward compatibility — existing code that calls `Row.new(row, formulae)` keeps working. Row-scoped formulae never touch `@namo`. Two-arity formulae use it when present. If a `Row` is constructed without a `Namo` and a two-arity formula is called, the `nil` produces a clear error rather than a missing argument error.

`Row#[]` dispatch for arity 1 and 2:

- 1: `proc.call(row)` — row-scoped
- 2: `proc.call(row, namo)` — collection-scoped

0.16.0: Parameterised formulae

Procs with arity > 2 receive `(row, namo, *extra_args)`. `Row#[]` forwards extra arguments:

```

e[:sma] = proc do |row, namo, field, period|
  window = namo[symbol: row[:symbol], date: ..row[:date]].last(period)
  window.sum{|r| r[field]} / window.length.to_f
end

```

```

row[:sma, :close, 20] # Row inserts self and namo, forwards :close and 20

```

`Row#[]` dispatch extended:

- 1: `proc.call(row)` — row-scoped
- 2: `proc.call(row, namo)` — collection-scoped
- >2: `proc.call(row, namo, *extra_args)` — parameterised

0.17.0: `Namo::Collection`

A hierarchical aggregate pattern for composing named `Namos`. `Namo::Collection < Namo` holds members — an `Array` of named `Namos` — and provides view methods that summarise or detail across them.

Use case: hierarchical budgets

The GT hypercar budget motivated this pattern. Sub-assemblies (`powertrain`, `chassis`, `body`, ...) are each `Namos` with shared columns (`:weight`, `:cost`, ...). The whole car is a `Collection` of these sub-assemblies, queryable both at summary level (“weight by assembly”) and detail level (“all line items across all assemblies”).

```

class Car < Namo::Collection
  def summary(dimension, by: :assembly, reducer: :sum)
    super
  end

  def detail(by: :assembly)
    super
  end
end

class SubAssembly < Namo; end

```

```

powertrain = SubAssembly.new(name: :powertrain, data: [...])
chassis    = SubAssembly.new(name: :chassis, data: [...])
body       = SubAssembly.new(name: :body, data: [...])

gt = Car.new
gt << [powertrain, chassis, body]
gt.summary(:weight).values(:weight).sum    # total weight by summing assembly summaries
gt.detail.values(:weight).sum              # total weight by summing every line item

```

The Car class overrides set by: :assembly as the per-class default. A bare `Namo::Collection.new` works equally well with by: passed at call sites.

Implementation

Lives at `lib/Namo/Collection.rb` per file convention.

State model: members are the substance, @data is a lazily-materialised view

A Collection’s substance is `@members`. The inherited `@data` is a *derived view* of those members, not independently-stored state. Any inherited row-operation (selection, projection, each, the set/composition/comparison operators, values) reads `@data`, so the Collection lazily materialises `@data` from `@members` — via the detail view — on demand. The user never has to call anything first: `collection[symbol: 'BHP']` just works, materialising detail under the hood.

The lazy view is **detail**, and the choice is principled rather than arbitrary. `detail` is the lossless view — the union of the members’ rows. `summary` is a *reduction*, something you explicitly ask for. A Collection’s rows simply *are* its members’ rows; the summary is a computed question you pose against them. So falling into the detail view on a bare row-operation is “treat the Collection as its line items,” while the summary is never reached by accident — only via `summary(...)` or `as_summary`.

Materialisation is **pure-live in 1.x**: every row-operation recomputes `@data` from `@members` via `detail`, with no memoisation. This matches 0.7.0’s discipline for values/coordinates — derived views recompute from current state, no caching. There is therefore no staleness problem: `<<` a member, then select, and the selection sees the new member because `@data` was just rebuilt. The cost of recomputing on every access is real but, per the 0.7.0 reasoning, invisible at typical sizes. Freeze-gated caching is a 2.x optimisation (see 2.x) and never changes this observable behaviour.

Because the lazy view is detail with the default `by: :member` (inject-iff-absent, below), the inherited operators behave correctly without ceremony — a `group_by`-derived Collection materialises with its intrinsic grouping dimension present (no injection), and an assembled Collection materialises with `:member` injected, exactly as an explicit `as_detail` would have produced.

View methods: summary, detail, as_summary, as_detail

The four view methods come in two pairs:

- `summary(dimension, by:, reducer:)` and `detail(by:)` are **non-mutating** — they return a fresh `Namo` derived from `@members`, leaving `@data` untouched. Use these when you want a view to *keep* — assign the returned `Namo` to a variable and operate on it independently of the Collection.
- `as_summary(dimension, by:, reducer:)` and `as_detail(by:)` are **mutating** — they set the Collection’s `@data` to the chosen view and return `self`, for an immediate fluent step. Because 1.x is pure-live, an explicit `as_*` set is *transient*: the next inherited row-operation re-materialises detail and overwrites it. So `as_summary` is for “be the summary for this immediate chain,” not for a persistent mode. To hold a summary, use the non-mutating `summary(...)` and keep its result.

The non-mutating pair is the primary interface. The mutating pair is the explicit, transient counterpart to the lazy detail materialisation — useful when you want a *non-default* tag (`as_detail(by: :assembly)`) or the

summary view for an immediate operation.

detail and the inject-iff-absent rule

`detail(by:)` unions the members' rows. Whether it injects the `by` dimension depends on whether that dimension already exists in the rows:

- If `by` is **already a dimension** in a member's rows, `detail` leaves the row untouched — the dimension is intrinsic, already present, no injection.
- If `by` is **not** a dimension, `detail` injects it (`row.merge(by => m.name)`) — promoting the member's name into a dimension.

This single conditional (`row.key?(by) ? row : row.merge(by => m.name)`) is the only place the two arrival paths — partition vs assembly — touch, and it's what makes the round-trip properties below hold.

Round-trip properties

A Collection can be arrived at two ways: by *splitting* a Namo (partition, via `group_by` at 0.18.0) or by *assembling* disparate Namos (`<<`). The two paths have different round-trip behaviour, and the difference is exactly whether the grouping axis is already a dimension.

Split round-trip — exactly idempotent. When a Collection comes from `group_by(:symbol)`, every member retains `:symbol` (it's a pre-existing dimension, the axis grouping happened *along*, not consumed by the split). So `detail(by: :symbol)` is a pure union — `:symbol` is already present, nothing injected — and reconstructs the original rows exactly:

```
namo.group_by(:symbol).as_detail(:symbol) == namo    # true (multiset equality, 0.6.0)
```

Assembly round-trip — idempotent only after the first flatten. When a Collection is assembled from independently-built members, those members do *not* carry a dimension identifying which is which — the identity lives in `member.name`. The first `detail(by: :assembly)` is the **dimension-creating** step: it injects `:assembly`, promoting the extrinsic member name into an intrinsic dimension. From that point the structure is in the intrinsic regime and every subsequent round-trip is exact:

```
collection.as_detail(:assembly)  # injects :assembly – now a real dimension, dimension count +1
  .group_by(:assembly)           # reconstructs members
  .as_detail(:assembly)          # union only – :assembly already present, exact
```

The promoted `:assembly` dimension is **retained, not projected away** — it's real data now (provenance of each row), and discarding it on a subsequent round-trip would lose information. Removal, if wanted, is an explicit contraction (`[-:assembly]`, 0.3.0), never an automatic side effect.

The invariant across both paths: **round-tripping never removes a dimension; the assembly path adds one (on the first flatten) and keeps it.** `as_detail(dim)` where `dim` is already present is a pure, reversible union; where `dim` is absent it is a one-way promotion that increases the dimension count by one.

Names, and unnamed members

A member's name is used for two things: `find(name)` lookup, and replace-by-name on `<<`. Neither is load-bearing for materialisation — `detail` unions rows and (for assembled Collections) injects the member name as a dimension, but the lazy `detail` and the round-trips work regardless of how names are populated.

Enforcement is therefore at the *point of use*, not at insertion. `<<` accepts any member, named or not — there is no insertion guard. The consequences of an unnamed member are simply the honest ones:

- `find(name)` matches on `member.name`; an unnamed member matches nothing and is never found. That is the truthful result of having no name, not an error.
- Replace-by-name engages only when the incoming member *has* a name that collides with an existing member's. Unnamed members always append (no name to collide on).

This is deliberately *not* an eager guard. An earlier draft raised `ArgumentError` on unnamed `<<`, but that guard fires on a condition that isn't always a mistake — `group_by` (0.18.0) constructs members named by their group value, and a nil-valued group key produces a legitimately nil-named member. Forbidding nil names eagerly would either break `group_by`'s nil-group case or force a sentinel. Moving enforcement to use-site avoids all of that: a nil-named group member is unfindable-by-name but its rows still materialise and round-trip correctly via the intrinsic grouping dimension, and a genuinely-forgotten assembly name surfaces as “not found” when you try to find it. This matches Namo's broader lazy, compute-on-access discipline — don't validate what you haven't been asked about.

Memoisation — deferred to 2.x

`find`, `summary`, `detail`, and the lazy `@data` materialisation are **not memoised in 1.x**. Every call recomputes from `@members`. This is the same pure-live discipline 0.7.0 applies to values/coordinates, held uniformly: 1.x is correctness and coherence, pure-live throughout.

Memoisation is a 2.x performance feature, opt-in via `freeze` and transparent (see 2.x's caching subsection). A user who never freezes a `Collection` gets pure-live recomputation forever; caching engages only on frozen instances, where immutability guarantees cached views stay correct. It never changes observable behaviour or liveness — it only changes cost, and only when the user has opted in by freezing.

Replace-by-name on <<

If a member with the same name is already present, `<<` removes the existing one before appending the new. Last-write-wins. This makes the `Collection`'s name `[]` member mapping a regular dictionary rather than a `multimap`, which is what users expect of a “hierarchy of named pieces.”

`<<` accepts a single member or an array of members (via `flatten`), so `gt << powertrain` and `gt << [powertrain, chassis]` both work.

Dependencies

`Namo::Collection` depends on:

- **Constructor widening (0.12.0)** — keyword `data:` for `Namo.new(data: [...])` in `summary` and `detail` (positional `Namo.new([...])` works equally for these two methods, so this is ergonomic rather than strict), and the `name:` attribute so members identify themselves and `find/replace-by-name` work on name. (Unnamed members are accepted but unfindable-by-name; see “Names, and unnamed members” above.)
- **<< with replace-by-name** — Collection-specific, defined here.

Two-arity formulae (0.15.0) and parameterised formulae (0.16.0) are not required by `Collection` but compose well with it: a `Collection`'s view can include a derived dimension that aggregates across the underlying members.

This dependency stack is why `Collection` lands at 0.17.0 rather than earlier. The handover explored pulling it forward, but it needs the constructor widening — keyword `data:` and `name:` (0.12.0) — and the `if name` subclass guard (0.12.0) to be in place first. Building it once against a complete substrate beats shipping it early and amending it across every release that fills in a dependency — including the memoisation that 2.x will add, which is cleaner to attach to a settled, pure-live 1.x `Collection` than to retrofit.

Tests

- `Collection.new.members == []`.
- A bare row-operation on a `Collection` lazily materialises detail — `collection[symbol: 'BHP']` works without a prior `as_detail`.
- Lazy materialisation is pure-live: `<<` a member, then `select`, and the selection reflects the new member.
- `<<` adds a member.

- `<<` with an existing-name member replaces.
- `<<` with an array adds each member.
- `<<` accepts an unnamed member (no error); the member is appended and is unfindable by name.
- `find(:name)` returns the member with that name, or nil.
- `find` on an unnamed member's would-be key returns nil (never matches).
- `summary(:weight)` returns a `Namo` with `{member: <name>, weight: <sum>}` rows.
- `summary(:weight, by: :assembly)` uses `:assembly` as the labelling dimension.
- `summary(:weight, reducer: :mean)` uses mean instead of sum (requires `:mean` method on `Array` — relies on user's `Statistics` gem or similar).
- `detail` injects the `by` dimension when absent from member rows.
- `detail` does **not** inject when `by` is already a dimension in member rows (intrinsic case) — rows pass through untouched.
- `find`, `summary`, `detail`, and lazy `@data` recompute live — no memoisation in 1.x (a mutation via `<<` is reflected on the next call).
- `as_summary` sets `@data` to the summary view and returns `self`.
- `as_detail` sets `@data` to the detail view and returns `self`.
- `as_*` is transient: after `as_summary`, a subsequent bare row-operation re-materialises detail (explicit views do not persist across operations in 1.x).
- After `as_summary` (immediately, before another operation), the `Collection`'s dimensions reflect the summary's columns.
- Assembly round-trip: `collection.as_detail(:assembly)` injects `:assembly` and the dimension is retained through a subsequent `group_by(:assembly).as_detail(:assembly)`.
- Subclass with default `by`: (like `Car` above) uses the override.

Documentation

- README section on `Namo::Collection`, with the GT-budget-style example.
- Note on the lazy detail materialisation (a `Collection` behaves as its detail view on any row-operation) and the pure-live, no-memo discipline.
- Note on the four view methods, the non-mutating/mutating split, and the transience of `as_*` in 1.x.
- Note on the inject-iff-absent rule and the two round-trip behaviours.
- Note on the `<<` replace-by-name semantics and the use-site (not insertion-time) treatment of unnamed members.
- Forward-note that freeze-gated memoisation is a 2.x optimisation, opt-in and transparent.

0.18.0: `group_by` returns a `Collection`

`Namo#group_by(dimension)` splits a `Namo` into a `Namo::Collection`, partitioning the rows by the values of the given dimension. This completes the `Enumerable` coherence pass begun at 0.11.0 — it is the one `Enumerable` method that produces row-shaped output and was not included there.

Why this is a separate release, and why it's here at all

`group_by` was originally scheduled for 2.x. It comes forward to 0.18.0 *because* 0.17.0 built `Namo::Collection` — its return type. The dependency is the whole story:

The 0.11.0 `Enumerable` pass shipped every method that returns a `Namo`. `group_by` was excluded not as a deferral but because it was structurally blocked: it returns a *partition* — a keyed set of sub-Namos — and the right type for that didn't exist. `{key => Array<Row>}` (Ruby's default) isn't in the `Namo` family, and `{key => Namu}` (a plain Hash of Namos) is a half-measure that doesn't carry the aggregation surface a partition wants. The honest type for “a `Namo` split into named pieces” is `Namo::Collection`, which is also the type for “named pieces assembled into a whole.” Assembly and partition are the same structure reached from opposite directions; `Collection` is the structure, and `group_by` is its partition-side constructor.

So `group_by` could not land until `Collection` existed. 0.17.0 built `Collection`; 0.18.0 is the first release in

which `group_by` is expressible. The two-release split (0.17.0 then 0.18.0) records the causality: `Collection`'s existence is what unblocks and brings forward `group_by`.

Behaviour

`group_by(dimension)` returns a `Collection` whose members are the groups — one member per distinct value of `dimension`, each a `Namo` holding the rows that share that value. Because `dimension` is a pre-existing dimension of the data, it is retained in every member's rows (the split happens *along* the axis, it does not consume it). This is what makes the split round-trip exactly idempotent with `as_detail` on the same `dimension`:

```
namo.group_by(:symbol)           # => Namu::Collection, one member per symbol
namo.group_by(:symbol).summary(:close, reducer: :mean) # mean close per symbol
namo.group_by(:symbol).as_detail(:symbol) == namo      # true – exact round-trip
```

Each group member is named by its group value (`find('BHP')` returns the BHP member), so the `Collection`'s `find` and the whole assembly API apply uniformly to a partitioned `Collection`. The group value is the member name — and because it was already a dimension in the rows, there is no extrinsic-key conflation: the name and the dimension value coincide by construction.

The `nil`-key case falls out cleanly from the use-site naming decision (0.17.0). If some rows have `dimension` missing or `nil`-valued, they form a group keyed by `nil`, and that member is named `nil`. There is no insertion guard to trip — `<<` accepts it — and the member is simply unfindable by `find` (you can't `find(nil)` meaningfully), but its rows still materialise into the detail view and round-trip correctly, because the grouping `dimension` (`nil`-valued for those rows, but present) is intrinsic. No rows are dropped, no sentinel is invented; the `nil` group is a first-class member that happens to have a `nil` name. This is exactly why 0.17.0 moved name-enforcement to use-site rather than guarding `<<`.

Relationship to the Enumerable pass

With `group_by` landing here, the coherence statement from 0.11.0 completes: every `Enumerable` method that produces row-shaped output returns a `Namo`-family type. The subset methods (`select`, `reject`, `sort_by`, `uniq`, `partition`, `take/drop`) return `Namo` (or `[Namo, Namu]`); `group_by` returns `Namu::Collection`. The family is `Namo` and `Namu::Collection`; the rule is uniform across both.

Implementation

```
def group_by(dimension)
  collection = Namu::Collection.new
  @data.group_by{|row| row[dimension]}.each do |value, rows|
    collection << self.class.new(rows, formulae: @formulae.dup, name: value)
  end
  collection
end
```

Each group is constructed as an instance of the receiver's class (subclass type preserved), carrying the parent's `formulae`, named by the group value so the `Collection` can key on it.

A block form (`group_by{|row| ...}` computing the group key from a block rather than naming a `dimension`) is a natural extension. Deferred consideration: the block form's groups are keyed by a computed value with no corresponding `dimension` in the rows, so the split round-trip would be the *assembly* case (the computed key must be injected as a `dimension` on `as_detail`), not the exact-idempotent case. Worth supporting, but the `dimension`-named form is the primary one and the one that round-trips cleanly.

Tests

- `group_by(:symbol)` returns a `Namu::Collection`.

- The Collection has one member per distinct value of `:symbol`.
- Each member holds exactly the rows matching its group value.
- Each member retains `:symbol` (the grouping dimension is not consumed).
- Each member is named by its group value (`find(value)` works).
- Each member carries the parent's formulae.
- Each member preserves the receiver's class (subclass type).
- Split round-trip: `namo.group_by(:symbol).as_detail(:symbol) == namo`.
- `group_by` on an empty Namo returns an empty Collection.
- `group_by(:sector)` where some rows have nil `:sector` produces a nil-named member holding those rows; no rows are dropped; the split round-trip still holds.
- `summary/detail` work on a `group_by`-derived Collection identically to an assembled one.

Documentation

- README section on `group_by` returning a Collection, with the round-trip example.
- Note that this completes the Enumerable coherence pass started at 0.11.0.
- Cross-reference to Collection (0.17.0) for the assembly side of the same structure.

1.0.0: Stable release

The 1.0 release includes everything through 0.18.0:

- Selection (exact, array, range, proc, regex), projection, contraction.
- Single-row formulae, two-arity formulae, parameterised formulae.
- The set algebra (+, -, &, |, ^).
- The composition algebra (*, **, /).
- The comparison algebra (==, ===, eql?, <, <=, >, >=), at both Namo and Row levels.
- Block forms on every operator that pairs rows.
- The inspection vocabulary (`dimensions`, `data_dimensions`, `derived_dimensions`, `coordinates`, `values`, `to_h`).
- Enumerable methods returning Namos for subset operations.
- Constructor widening (keyword `data:` and `name:`) and polymorphic `[]`.
- `Namo::Collection` for hierarchical aggregates, reachable by assembly (`<<`) or partition (`group_by`).
- `group_by` returning a Collection, completing the Enumerable coherence pass.

This is the correct, tested, conservative foundation. No metaprogramming magic, no `method_missing`, no `instance_eval`. Formulae work via `e[:name] = proc{|row| row[:close] / row[:book_value]}` — clear, explicit, proven.

1.0 is the set of features that are well-understood, thoroughly tested, and unlikely to change.

Estimated performance: ~0.3s for a 2,000 row daily trading screen with indicators and scoring. Pure Ruby, no native dependencies. Adequate for interactive use and daily batch jobs. Not suitable for datasets over ~50,000 rows without patience.

1.1: Benchmarking suite

Performance and stress-testing infrastructure. Establishes baselines for every feature so that subsequent optimisations can be measured against concrete numbers.

The suite should cover:

- Construction: `Namo.new` from N rows, measuring hash ingestion and dimension/coordinate inference.
- Selection: single value, array, range, proc, regex, at various dataset sizes.
- Projection and contraction: `[]` dispatch overhead.
- Formulae: row-scoped resolution, formula chains (A references B references C), two-arity formulae, parameterised formulae.

- Enumerable: each, map, select, reduce, max_by — measuring Row object creation per iteration.
- Set operators: +, -, &, |, ^ at various dataset sizes.
- Composition: * across different dimension overlaps.
- Collection: <<, find, summary, detail at various member counts and per-member row counts.

Each benchmark at multiple scales — 100, 1,000, 10,000, 100,000, 1,000,000 rows — to show where curves bend and where Ruby’s overhead dominates.

Use benchmark-ips for iterations-per-second measurements. Results should be recorded and versioned so regressions are visible across releases.

The 1.1 numbers become the baseline for 2.x comparisons.

1.2: Loaders

Optional requires for common data sources. No loader loaded by default. No dependencies added to Namo core.

```
require 'namo/loaders/csv'
require 'namo/loaders/sequel'
require 'namo/loaders/json'
```

Each extends Namo with a class method (from_csv, from_sequel, from_json). Loaders are sugar — the constructor already takes arrays of hashes, so loading is always possible without them.

2.x: Bare names and Ruby-side optimisation

Theme: the expressive leap.

The 1.x / 2.x boundary is a discipline boundary as much as a feature one. **1.x is correctness and coherence, and pure-live throughout** — every derived view (values, coordinates, to_h, and Collection’s find/summary/detail/lazy @data) recomputes from current state on every access, with no memoisation anywhere. Liveness is unconditional; there is no staleness to reason about. **2.x is the performance phase**, where memoisation lands across the board, alongside bare names, method_missing, DefineAccessors, and Finite. Caching is introduced uniformly here rather than piecemeal in 1.x, so the rule is simple to state and simple to trust.

Estimated performance: method_missing on Row adds ~5-10% overhead versus 1.x on first access per dimension per Row. The DefineAccessors optimisation (below) recovers most of this by defining real methods on first access. After warm-up, 2.x should be within ~2-3% of 1.x. Pure Ruby performance tuning against the 1.1 benchmarking suite may yield further gains. All estimates — actual numbers depend on the 1.1 baseline.

From 2.0, require 'namo' loads bare name resolution by default. The full experience is the default. Users who want the 1.x explicit style can opt out:

```
require 'namo'           # 2.x default: bare names included
require 'namo/core'      # opt out: 1.x explicit style, no method_missing on Row
```

Bare name resolution

Row resolves dimension names and formulae as bare method calls via method_missing, delegating to self[name, *args]. This eliminates hash access syntax from formulae:

```
# 1.x style
e[:price_to_book] = proc{|row| row[:close] / row[:book_value]}

# 2.x with bare names via `method_missing` on Row
e[:price_to_book] = proc{|row| row.close / row.book_value}
```

```
# 2.x with bare names via `instance_eval` for arity-0 procs
e[:price_to_book] = proc{ close / book_value }
```

Bare names also work in cross-row formulae. Fixed dimension references resolve against the current Row. Variable field names (passed as parameters) use Symbol#to_proc via &field:

```
e[:sma] = proc do |row, namo, field, period|
  window = namo[symbol: symbol, date: ..date].last(period)
  window.sum(&field) / window.length.to_f
end
```

Here symbol and date are bare names resolving against the Row. &field uses Symbol#to_proc to send the field name to each Row in the window.

Resolution order in Row#method_missing: formulae first, then row data, then super (raises NoMethodError for typos).

method_missing on Namo for formula assignment

Namo resolves formula assignment as bare method calls via method_missing, catching name= calls:

```
e.price_to_book = proc{ close / book_value }
e.sma = proc do |row, namo, field, period|
  # ...
end
```

Module-based formula libraries (documented pattern)

With bare name resolution in place, formulae can be defined as plain Ruby methods in modules and included into Namo subclasses. This is not a Namo feature — it's standard Ruby. It's documented here as a recommended pattern:

```
module Indicators
  def sma(row, namo, field, period)
    window = namo[symbol: symbol, date: ..date].last(period)
    window.sum(&field) / window.length.to_f
  end

  def change
    close - open
  end

  def earnings_yield
    eps / close
  end
end

module Scoring
  def value_score
    return 0 unless pe && pe > 0
    pe < 10 ? 2 : pe < 15 ? 1 : pe < 25 ? 0 : -1
  end

  def total_score
    value_score + book_score + yield_score + momentum_score + trend_score
  end
end
```

```

def action
  s = total_score
  s >= 5 ? 'BUY' : s >= 2 ? 'WATCH' : s >= 0 ? 'HOLD' : total_score <= -2 ? 'SELL' : 'AVOID'
end
end

class TradingAnalysis < Namo
  include Indicators
  include Scoring
end

```

Scoring strategies are swappable via dependency injection.

Namo#=== revision under module-based formulae

When module-based formulae become a real pattern (rather than a documented one), `Namo#===` (and `Namo#eql?`, and `Namo#hash`) must be revised to enumerate module-defined method names alongside `@formulae.keys`. The 0.6.0 implementation compares formula *names* via `@formulae.keys.sort` — which is correct for `[]`-registered formulae, but module-defined formulae don't enter `@formulae`. They're plain Ruby methods on the included module, discovered by Row's `method_missing` through the normal method-lookup chain. So a `TradingAnalysis` instance with `include Indicators` would have `@formulae.keys == []` even though it has the full `Indicators` analytical surface; under 0.6.0's `===`, two such instances are correctly `===` to each other (same dimensions, same — empty — `@formulae.keys`), but they'd also be `===` to a vanilla `Namo` of the same dimensions, which is wrong: a vanilla `Namo` has no indicators.

The starting point

```

# 0.6.0 implementation
def ==(other)
  return false unless other.is_a?(Namo)
  dimensions.sort == other.dimensions.sort &&
    @formulae.keys.sort == other.formulae.keys.sort
end

```

Correct for the case where all analytical structure flows through `@formulae`. The gap is everything that lives on the class instead of the instance.

Alternatives considered Three candidates were on the table:

Path A: Check class identity. `===` also requires `self.class == other.class`, matching `eql?`'s class-strictness. Same class implies same included modules implies same module-defined formulae, by transitivity. Simple to implement but rejects useful equivalences — two `Namos` from different classes with the same queryable surface (one via `[]`, one via `include`) would not be `===` even though they behave identically as analytical artefacts.

Path B: Duck-type the queryable surface. `===` compares the full set of names queryable on each `Namo`, regardless of where each name's definition lives. Storage dimensions, instance `@formulae` keys, and class-level analytical method names all contribute. Two `Namos` with the same queryable surface are `===` regardless of class or how their formulae were registered. Matches the design philosophy's unified-treatment principle: care about what's queryable, not where the definition lives.

Path C: Class as proxy for analytical surface. Same code as Path A, framed differently — class identity stands in for the union of all analytical methods. This is just Path A with different rationale; the behaviour is identical and the same objection applies.

Path B is the right answer. It honours duck-typing across class boundaries, handles the `[]`-vs-`include` mixture correctly, and doesn't change behaviour when users refactor a formula from per-instance assignment

to a module method.

Path B implementation sketch

```
def ==(other)
  return false unless other.is_a?(Namo)
  queryable_names == other.send(:queryable_names)
end

private

def queryable_names
  (dimensions + @formulae.keys + class_formula_names).to_set
end
```

`to_set` gives order-insensitive comparison; `.sort` would also work and avoids the Set allocation. Either is fine — pick when implementing. The equivalent changes to `eql?` and `hash` follow the same shape: substitute `queryable_names` for `@formulae.keys.sort` everywhere they appear.

What `class_formula_names` returns The open design question. Two candidate mechanisms:

Implicit identification. All public instance methods on the `Namo` subclass that aren't inherited from `Namo` itself are treated as formulae. Simple, no declaration overhead. Cost: conflates analytical methods with any custom method — a `def to_s` override on the subclass would be misread as a formula.

Explicit declaration. Modules opt into being treated as formula libraries, either by a marker module (`include Formula::Module`) or per-method declaration (`formula :sma`). Cost: declaration overhead. Benefit: precise — only intentionally-analytical methods count.

The implicit approach is more ergonomic; the explicit approach is more robust. Choose based on what the module-based formula mechanism actually looks like once it's implemented.

Body equality is not part of any equality operator 0.6.0 already settled this for the instance-level case: `==`, `eql?`, and `hash` all compare formula *names* (`@formulae.keys.sort`), not the procs themselves. The Path B revision extends that stance to module-defined formulae — adding more names to compare, not adding proc-body comparison. The rationale is the same as at 0.6.0 design time:

1. **Proc identity isn't function identity.** Two interactively-built `Namos` defining `:doubled = proc{|r| r[:x] * 2}` separately have different proc objects. Comparing by proc identity would call them unequal despite identical behaviour. Users would be surprised.
2. **`==` is a pattern-match operator, not a behaviour-identity operator.** Pattern-matching asks “does this candidate fit this pattern?” The pattern is the analytical shape — what names are queryable. `eql?` makes the same call for the same reason: in Ruby there's no cheap, reliable way to compare two procs for behavioural equivalence, so neither operator pretends to.

The deeper question of true behavioural equivalence (AST comparison, `iseq` comparison, DSL normal forms, etc.) is its own significant project; see the “Proc equivalence options” note in the project memory for the landscape. Until one of those paths lands, key-based comparison is the honest pragmatic position.

When this revision lands In whichever 2.x release introduces module-based formula registration as a real mechanism (rather than just a documented pattern using plain Ruby methods). Until then, the 0.6.0 key-based implementation handles `[]`-registered formulae correctly; the gap (module-defined methods) only matters once those methods actually exist as a registration channel.

DefineAccessors optimisation

Inspired by Hashie’s pattern — lazily define real methods on first access to avoid repeated `method_missing` overhead:

```
class Row
  def method_missing(name, *args)
    if @formulae.key?(name)
      define_singleton_method(name) do |*a|
        resolve(name, *a)
      end
      send(name, *args)
    elsif @row.key?(name)
      define_singleton_method(name) { @row[name] }
      @row[name]
    else
      super
    end
  end
end
```

First access fires `method_missing` and defines a real method. Every subsequent access is a direct method call. Consider when profiling shows `method_missing` as a bottleneck.

Hashie as prior art, not a dependency

Hashie (particularly `Hashie::Mash`) is the primary prior art for method-style hash access and coercion in Ruby. Namo’s `Row` and the coercion module design draw from Hashie’s patterns. However, Hashie should not be used as a dependency or as internal storage for Namo:

- Namo already handles method-style access through `Row`’s `method_missing`. Wrapping internal hashes in `Hashie::Mash` would add a second `method_missing` layer — double the dispatch overhead for no additional capability.
- Hashie does key normalisation (string/symbol indifferece) and deep conversion of nested hashes. Namo doesn’t need either — keys are always symbols and data should be flat.
- Hashie has a documented history of subtle breakage, particularly around `respond_to_missing?` interactions with other gems and method name collisions with Ruby built-ins.
- The performance cost of `Mash` allocation over plain `Hash` allocation is measurable.

Pure Ruby performance tuning

Profile against the 1.1 benchmarking suite. Identify and address hot paths within pure Ruby: `Row` object allocation, formula chain resolution, selection dispatch. No native code — just better Ruby.

Caching / memoisation (opt-in via freeze, transparent)

1.x is pure-live: values, coordinates, `to_h`, and `Collection`’s `find/summary/detail/lazy @data` all recompute on every access. 2.x adds memoisation as a performance layer, with two non-negotiable properties:

1. **Transparent.** Memoisation never changes observable behaviour, output, or liveness. Same inputs produce same outputs with the same currentness guarantees; only the cost differs. A user never has to reason about whether a cached value is stale — that is the cache’s problem, and if it can’t be guaranteed correct, the cache does not engage.
2. **Correct by construction, opt-in via freeze.** Caching engages only on **frozen** instances. An unfrozen `Namo` or `Collection` can mutate (`[]=`, `<<`, `@data/@members` changes), so its derived views could go stale

under a cache — therefore an unfrozen instance always recomputes live, exactly as in 1.x. A frozen instance cannot change, so a cached derived value stays correct forever. The rule:

- **Unfrozen** □ pure-live, always recompute, no caching. (The 1.x default, unchanged.)
- **Frozen** □ caching may engage, because immutability guarantees freshness.

This makes memoisation genuinely optional without a flag, a mode, or a config the user must learn. The user opts in by doing the thing they would do anyway when finished mutating — `freeze` for sharing, hash-keying, or thread-safety (see the Mutability open question) — and the caching rides along for free. A user who never freezes never caches and never thinks about it. The optionality is about *whether caching engages* (the freeze decision), never about *whether a cache is correct* — a cache only ever exists on immutable state, so stale-cache wrong answers are impossible by construction.

Scope of the 2.x caching work:

- **Namo inspection aspects** — memoise values/coordinates/to_h on frozen Namos (the “wholesale aspect caching” the 0.7.0 note defers to here).
- **Collection views** — memoise `find/summary/detail/@data` materialisation on frozen Collections. This is the memo-and-invalidation apparatus that was deliberately *removed* from the 0.17.0 implementation; it returns here, where it is measured against the 1.1 benchmark suite and justified — and where freeze makes invalidation-on-mutation unnecessary (a frozen Collection never mutates, so there is nothing to invalidate).

Contingency: this work is gated on the **freeze semantics**, currently in the Open Questions section rather than a scheduled release. Caching attaches to freeze, so resolving the freeze semantics is a prerequisite for the caching workstream. If freeze lands before 2.x’s performance phase, caching can attach to it directly; if the freeze semantics are still open when this work begins, settling them comes first.

Bare-name ergonomics on Collections

`group_by(:symbol)` returns a `Namo::Collection` from 0.18.0; the return type is settled in 1.x. What 2.x adds is the bare-name resolution that makes aggregation over a Collection’s members read cleanly:

```
# 1.x – works, but explicit
namo.group_by(:symbol).summary(:close, reducer: :mean)
# => Namu with {symbol:, close:} rows

# 1.x – explicit member-wise computation
namo.group_by(:symbol).members.map{|n| n.values(:close).sum / n.length}

# 2.x – bare names make member-wise computation read cleanly
namo.group_by(:symbol).members.map{|n| n.close.sum / n.length}
```

The return-type change (`group_by` □ `Collection`) is *not* a 2.x concern — it landed at 0.18.0, gated on `Collection` at 0.17.0. 2.x only contributes the bare-name reading on the resulting members. The conceptual model was already unified in 1.x: assembly and partition both produce a `Collection`; bare names make the post-partition computation as ergonomic as the pre-partition selection.

Finite module

A module that includes `Enumerable` and adds `last` and `reverse_each` for finite collections. Default implementation uses `entries`; `Namo` overrides for performance by going straight to `@data`.

```
module Finite
  include Enumerable

  def last(n = nil)
    n ? entries.last(n) : entries.last
  end
end
```

```

end

def reverse_each(&block)
  return enum_for(:reverse_each) unless block_given?
  entries.reverse_each(&block)
end
end

```

Finite is a standalone concept — potentially a separate gem. Any finite Enumerable can include it.

Namo includes Finite and overrides `last` to go straight to `@data`, wrapping results in Rows. The Row constructor's `@namo` parameter (from 0.15.0) is passed through so Finite-wrapped Rows participate in two-arity formulae just like rows from `each`:

```

def last(n = nil)
  if n
    @data.last(n).map{|row| Row.new(row, @formulae, self)}
  else
    Row.new(@data.last, @formulae, self)
  end
end

```

Finite lands in 2.x rather than 1.x because it's primarily a performance optimisation — the 1.0 `last(n)` works correctly via the Enumerable default, just less efficiently. 2.x is the dedicated performance phase, and Finite belongs there.

Finite as a separate gem

Extract the Finite module into a standalone gem for use by any finite Enumerable, independent of Namu. Lands after the in-Namu version is stable.

3.x: DSL, columnar storage, and C acceleration

Theme: the optimised engine.

Estimated performance: columnar storage accelerates selection-heavy workloads — scanning one contiguous array versus touching every row's hash. C extension via `RubyInline` targets the remaining hot paths (iteration, row creation). Combined, the 2,000 row daily screen should drop from ~0.3s to ~0.1s. Selection on large datasets (100,000+ rows) should see the biggest improvement from columnar storage. The C extension adds ~4x speedup on tight numeric loops but the bottleneck is object allocation, not arithmetic — expect modest overall gains from C alone. All estimates.

From 3.0, require `'namo'` loads both bare names and the DSL block syntax by default. Users can opt out at any granularity:

```

require 'namo'           # 3.x default: bare names + DSL
require 'namo/core'      # 1.x explicit style only
require 'namo/bare_names' # bare names without DSL
require 'namo/dsl'       # DSL block syntax without bare names
require 'namo/dsl/bare_names' # both (same as default, explicit about it)

```

The DSL and bare names are independent features on different objects — the DSL uses `method_missing` on a builder to catch formula registrations, while bare names use `method_missing` on Row to resolve dimension access. The DSL block works without bare names (procs use explicit `row[:close]` inside), and bare names work without the DSL block (formulae registered via `e.name = proc{}`). In practice, most users will want both.

DSL block syntax

A minimal-ceremony specification syntax using a builder with `method_missing`:

```
namo do
  sma          ->(field, period) { window(field, period).mean }
  change       -> { close - open }
  price_to_book -> { close / book_value }
  earnings_yield -> { eps / close }
  golden_cross  -> { sma(:close, 20) > sma(:close, 50) }
  value_score   -> { pe < 10 ? 2 : pe < 15 ? 1 : pe < 25 ? 0 : -1 }
  total_score   -> { value_score + book_score + yield_score + momentum_score + trend_score }
  action        -> { total_score >= 5 ? 'BUY' : total_score >= 2 ? 'WATCH' : total_score >= 0 ? 'HOLD' : total
2 ? 'SELL' : 'AVOID' }
end
```

The builder's `method_missing` catches each name-lambda pair. No `def`, no `end`, no `module`, no `class`, no `e.name = proc`. Just names and computations.

Columnar storage

A columnar backend (hash of arrays) to accelerate selection and aggregation for pure-Ruby workloads. The design uses a mezzanine pattern — `Namo` delegates to `Namo::RowStore` or `Namo::ColumnStore` with the same public API.

This section is about *internal storage*, not about adding new public methods. `to_h` and `values` (both producing columnar output) already exist from 0.7.0; what 3.x changes is the cost of computing them. With row-oriented storage (current), `to_h` builds the columnar hash on demand by iterating rows. With columnar storage, `to_h` returns the internal representation directly. Same output, different cost.

The columnar form is exactly the shape that `to_h/values` produces — `{symbol: ['BHP', ...], close: [42.5, ...]}`, a hash of arrays. Columnar storage means accepting that shape as input and using it as the internal representation. The round-trip is clean: `Namo.new(namo.to_h)` reconstructs from columnar output.

Development order:

1. Default (current): always `RowStore`, no choice, no configuration.
2. Choose at instantiation: `Namo` detects the input shape (array of hashes vs hash of arrays) or accepts an explicit `storage:` keyword.
3. Dynamic: one primary layout with the other cached on demand, cache invalidated on mutation.

```
# Row-oriented (array of hashes, current)
namo = Namo.new([{:symbol: 'BHP', close: 42.5}, ...])
```

```
# Column-oriented (hash of arrays) – the shape to_h produces
namo = Namo.new({symbol: ['BHP', ...], close: [42.5, ...]})
```

When DuckDB is the backend (4.x), neither layout matters — the data lives in DuckDB. Columnar storage only matters when `Namo` is doing the computation itself in pure Ruby.

C extension via RubyInline

Optional C acceleration for hot paths via `RubyInline` (`zenspider`). Targets: selection (linear scan over contiguous array), iteration (row creation), and storage layout. Formula resolution stays in Ruby.

Pure Ruby is the default. C extension is an optional `require`. The API doesn't change:

```
require 'namo'
require 'namo/ext' # optional C acceleration, falls back to pure Ruby if unavailable
```

Coercion modules

Two forms of coercion, both declared as includable modules:

Ingestion coercion — fix types from CSV/JSON:

```
module OHLCVTypes
  def self.included(base)
    base.coerce :date, Date
    base.coerce :close, Float
    base.coerce :volume, Integer
  end
end
```

Dimension alignment coercion — map dimensions for * composition:

```
module TemporalAlignment
  def self.included(base)
    base.coerce :date, to: :quarter do |date|
      "#{date.year}-Q#{((date.month - 1) / 3) + 1}"
    end
  end
end
```

Coercion modules can be placed on the analysis class, on a loader, or on an ad hoc instance. Namo doesn't have an opinion about where they live. Inspired by Hashie's coercion extensions, but applied at the dimensional level.

The two forms are distinguishable by signature:

- `coerce :date, Date` — ingestion, fix the type of this dimension's values.
- `coerce :date, to: :quarter do ... end` — alignment, map this dimension to another for *.

Nested data flattening

Data from JSON APIs and other sources may arrive with nested structures:

```
{symbol: 'BHP', fundamentals: {eps: 1.2, pe: 14.5}}
```

Nested values break Namo's foundational principle — every key is a dimension. The solution is flattening at ingestion via a coercion declaration:

```
base.coerce :fundamentals, extract: [:eps, :pe, :book_value]
```

Which pulls specified keys up to the top level and drops the wrapper. The result is a flat hash where everything is a dimension.

Conversion discovery on *

When * finds no exact dimension name match, it checks the class's coercion registry for declared conversion paths. This replaces the earlier `respond_to?` design with `explicit`, `inspectable`, `testable` declarations.

Co-incident: Python bridge Paths 1 and 2 (separate repo)

Developed in a separate `namo-python` repository. The Python user writes Namo's Ruby DSL inside a triple-quoted string. The 3.x DSL block syntax is what they see:

```
from namo import Namo

analysis = Namo.define("""
  e.golden_cross = proc{ sma(:close, 20) > sma(:close, 50) }
```

```

e.earnings_yield = proc{ eps / close }
e.action          = proc{ total_score >= 5 ? 'BUY' : total_score >= 2 ? 'WATCH' : 'HOLD' }
""")

```

```
results = analysis.run(data)
```

Path 1 (shell out): Python wrapper writes the DSL to a temp file, invokes ruby via subprocess, parses JSON results from stdout. Requires separate Ruby installation. ~0.5s overhead per invocation.

Path 2 (rb_call): MessagePack RPC to a persistent Ruby process. Lower latency (~50ms per call), more complex setup. Still requires Ruby installed separately.

Both paths build against 3.x Ruby Namo. The Python user experiences the full expressiveness of the DSL syntax. Ships with the Indicators module as a freebie so Python users get immediate value.

4.x: mRuby, SQL pushdown, and DuckDB

Theme: the database-integrated engine.

Estimated performance: DuckDB pushdown is the architectural win. Window functions (SMA, EMA) on millions of rows execute at ~0.3s in DuckDB versus ~45s in pure Ruby. The 2,000 row daily screen drops to ~0.05-0.1s total — DuckDB handles the heavy computation, Ruby handles the scoring and selection on the small result set. For the full 8.8M row ASX history, DuckDB makes it feasible where pure Ruby never could. Competitive with Polars (~0.1s) on equivalent workloads. All estimates.

mRuby compatibility

Namo's core compiled under mRuby. This enables embedding Namu in other environments without a full CRuby installation. Connects to existing mRuby compatibility work (e.g. `stateful.rb` 2.1.0).

DuckDB integration

DuckDB as a high-performance backend via Sequel or the `duckdb` gem. Four paths were considered, ordered from most manual to most automatic:

Path A (rejected): Write raw SQL for large slabs of functionality. Rejected — means maintaining SQL alongside Ruby, and the two representations can drift apart.

Path D (immediate): Use Sequel to generate DuckDB window functions before Namu ingests the data. No changes to Namu.

Path C (next): Formulae carry SQL metadata alongside the Ruby implementation. Namu detects the backend and inlines SQL where available:

```

module Indicators
  def sma(row, namo, field, period)
    window = namo[symbol: symbol, date: ..date].last(period)
    window.sum(&field) / window.length.to_f
  end

  SQL = {
    sma: ->(field, period) {
      "AVG(#{field}) OVER (PARTITION BY symbol ORDER BY date ROWS BETWEEN #{period -
1} PRECEDING AND CURRENT ROW)"
    }
  }
end

```

Path B (eventually): Namo generates SQL from formula definitions automatically.

Different Namos in the same composition can use different backends. OHLCV from DuckDB with SQL pushdown, fundamentals from Sequel, macro data from CSV. They compose via `*` regardless of origin.

SQL pushdown from formula metadata

Formulae declare their SQL equivalents. Namo uses the SQL version when a database backend is detected and falls back to Ruby when it can't. The Ruby implementation is the truth. The SQL is the optimisation. Both are declared in the same place.

Co-incident: Python bridge Path 3 (separate repo)

With mRuby compatibility in 4.x, the Python bridge can embed the mRuby interpreter directly. `pip install namo` works with no separate Ruby dependency. Built in the `namo-python` repo against the 4.x mRuby-compatible core.

Near-zero call overhead — in-process, no serialisation. Cross-platform binary distribution needed (wheels per platform).

5.x: Streaming, transpilation, and genetic algorithms

Theme: the analytical platform.

Estimated performance: streaming eliminates the memory constraint — 8.8M rows never loaded simultaneously. Processing cost is per-window rather than per-dataset. Transpilation to SQL or Python/Polars delegates execution entirely to native engines — Namo becomes a specification tool with near-zero runtime cost. GA support benefits from DuckDB's speed on each candidate evaluation. Performance is bounded by the backend, not by Namo. All estimates.

Streaming / progressive computation

For large datasets (8.8M+ rows), process data in windows rather than loading everything. Rolling buffer per symbol, incremental indicator computation, results retained, raw data discarded.

A streaming Namo stays unfrozen — it accepts new rows, invalidates caches on mutation, and reflects current state. The analytical case is a Namo that gets frozen after setup.

Transpilation

Namo as a specification language that emits executable code in other targets:

- SQL (DuckDB, PostgreSQL) for production deployment.
- Python/Polars for handoff to Python teams.

Explore interactively in Ruby. Deploy as transpiled output. The exploration tool and the production artifact are the same object viewed from different angles.

Genetic algorithm support

Parameterised formulae enable GA-based strategy optimisation:

```
namo do |params|
  value_score -> { pe < params[:pe_strong] ? 2 : pe < params[:pe_good] ? 1 : 0 }
  action      -> { total_score >= params[:buy_threshold] ? 'BUY' : 'HOLD' }
end
```

Genome is a hash of parameters. Fitness is backtest return. Mutation/crossover operate on hashes. DuckDB evaluates each candidate at native speed.

Speculative / unversioned

These options are not scheduled. They become relevant when need and adoption justify the investment.

Rust acceleration via Rutie + Thermite

Rust accelerates specific hot paths within Ruby-side Namo. Ruby remains the host. The native code is an optimisation layer underneath the Ruby API. Formula resolution, `method_missing`, and DSL stay in Ruby.

Rutie provides Ruby bindings for Rust via macros. Thermite handles compilation and gem distribution with pre-compiled binaries. ~20x speedup observed on parsing benchmarks.

Targets: selection, iteration, storage layout. The same hot paths as the C extension (3.x) but with Rust's memory safety and the Rust ecosystem's momentum in data tooling (Polars, DuckDB, Apache Arrow).

```
namo/  
  lib/  
    namo.rb          # pure Ruby, always works  
    namo/ext.rb      # loads native extension, falls back to pure Ruby  
  ext/  
    namo_ext/  
      Cargo.toml     # Rust project  
      src/  
        lib.rs       # Rust implementations of hot paths  
  Rakefile           # Thermite tasks for building
```

Work on Rust acceleration builds familiarity and infrastructure that would feed into the Rust-native engine if it ever happens.

FFI (generic)

Compile any language (Rust, C, Crystal) as a C-compatible shared library (`cdylib`), load via Ruby's `ffi` gem or `stdlib Fiddle`. More boilerplate than Rutie but language-agnostic — the same FFI interface works regardless of what produced the shared library.

Rust-native engine with Ruby DSL parser

Namo's core reimplemented in Rust with a parser for the Ruby DSL syntax. The DSL specification strings are parsed, not evaluated — no Ruby runtime at all. The Rust engine is language-agnostic and can be exposed to any host language via its native FFI mechanism:

- Python via PyO3 (the same way Polars works).
- Ruby via Rutie (replacing pure Ruby internals with Rust for speed).
- Node.js via `napi-rs`.
- Go via CGo.
- Any language via C-compatible FFI.

The Ruby DSL becomes a portable notation — a specification language that any host can submit to the Rust engine. The engine parses it, builds the internal representation, and executes it at native speed.

This path also connects to the transpilation story — if the Rust engine can parse the DSL, it can also emit SQL, Python/Polars expressions, or other output formats from the same parsed representation.

Not constrained to any single language. Largest engineering investment. Only justified if Namo achieves significant adoption across multiple language communities.

WASM compilation

Compile the Rust-native engine or mRuby core to WebAssembly for browser execution. Enables Namopowered analysis in client-side web applications without a server.

Open questions

Mutability

Namo currently has `attr_accessor :data, :formulae`, making instances fully mutable. This conflicts with caching (stale results), thread safety (race conditions), and the principle that formulae are declarations.

The proposed model: Namos are mutable by default for interactive exploration. Call `freeze` when the shape is settled. Ruby's built-in `freeze` semantics apply — mutation methods (`[]=`, `.name=`) check `frozen?` and raise `FrozenError` if so. Caching is safe on frozen Namos. Threads can share frozen Namos.

```
namo = TradingAnalysis.new(ohlcv_data)
namo.change = proc{ close - open }
namo.score = proc{ ... }
```

```
# exploration done
namo.freeze
```

```
# now cacheable, shareable, immutable
namo.score = proc{ ... } # => FrozenError
```

The streaming case is simply a Namop that never gets frozen. No special class needed — the distinction between analytical and streaming is a lifecycle difference, not a type difference. `freeze` is the transition point.

`attr_accessor` should become `attr_reader` on the public API, with mutation going through methods that check `frozen?`. Internal state (`@data`, `@formulae`) should be frozen when the Namop is frozen.

The `eql?`/`hash` contract introduced in 0.6.0 reinforces this. `eql?` requires `hash` to be consistent — `a.eql?(b)` implies `a.hash == b.hash` — and `hash` is computed from `@data`, `class`, and `@formulae`. Any mutation changes `hash`, which means an unfrozen Namop cannot be safely used as a Hash key or Set member: lookup might miss the entry the Namop was stored under because the Namop's `hash` has shifted. Frozen Namos have stable hashes and are safe for hash-keyed lookup. The lifecycle pattern (mutate during exploration, `freeze` before sharing) maps directly onto when `eql?`-based collection operations become reliable.

Live data and cache invalidation

For streaming/live Namos that stay unfrozen, formula results cached on Rows become stale when new data arrives. Options:

- No caching on unfrozen Namos (simple, slower).
- Generation counter — each mutation increments a counter, cached results carry the generation they were computed at, stale results are recomputed.
- Event-based invalidation — mutation notifies dependents.

The simplest approach (no caching when unfrozen) is probably right for the first implementation. Optimise if profiling shows it matters.

Coercion design

The coercion section (under 3.x) proposes a dual-purpose `coerce` keyword for both ingestion type fixing and dimensional alignment. Open questions:

- Is a single keyword for two different operations clear or confusing?

- Should ingestion coercion and alignment coercion be separate APIs?
- When multiple alignment coercions exist for a dimension (e.g. `:date` can coerce to both `:quarter` and `:month`), how does `*` choose? Raise ambiguity error? Accept a preference list?
- Should alignment coercion include a strategy (e.g. “most recent before”) or just a value transformation?
- How does coercion interact with `freeze`? Are coercion declarations frozen with the `Namo`?
- Should nested data flattening (`extract:`) be part of the coercion API or a separate ingestion concern?
- Should flattening be recursive (nested hashes within nested hashes) or single-level only?

Formula shadowing and method collisions

If a formula and a data dimension share the same name, which wins in Row resolution? Currently formulae take priority. Should this be configurable? Should it warn?

Related: Hashie::Mash’s long history of problems with method names that collide with Ruby built-ins (`class`, `hash`, `sort`, `zip`, `count`) is directly relevant. Row’s `method_missing` has the same risk — a dimension named `class` or `method` would shadow Ruby’s built-in methods. Options:

- Maintain a safelist of Ruby method names that `method_missing` won’t intercept.
- Warn when a dimension name collides with a built-in.
- Always allow hash-style access (`row[:class]`) as a fallback when bare name access is blocked.
- Document the risk and accept it — dimension names like `class` and `method` are unlikely in practice.

Operator return types with subclasses

When `TradingAnalysis * Namu` is evaluated, what class is the result? `TradingAnalysis` (preserving the included modules)? `Namu` (the base class)? The left operand’s class? This affects whether formulae from included modules are available on the composed result.

0.6.0 settles part of this question for equality: `eq?` cares about class match (`TradingAnalysis.new(data).eq?(Namu.new(data))` returns false even if the data matches), `==` does not. 0.12.0’s subclass guard pattern (`if name in initialize`, introduced with the `name: attribute`) addresses the side-effects-on-operator-results question — operator-derived instances are name-less and skip side effects. The class of operator results currently defaults to the receiver’s class, which works for same-class composition. Cross-class composition (`TradingAnalysis * SectorMetrics`) still raises the question of which subclass’s modules carry through.

Presentation examples

See `EXAMPLES.md` for full four-stage progressions (competitor tool □ 1.x □ 2.x □ 3.x) across seven disciplines with side-by-side code comparisons.